

FPC 3.2.2 Referenz Handbuch

Inhaltsverzeichnis

Lizenz - Bitte lesen !!!	3
Überblick	5
GNU Dokumentation	6
GNU - Manifest	7
FS und OSS	14
Lizenzen - 'LGPL'	19
FAQ - Questions-Answers	21
1 - Basic Questions	24
Lizenzen - MIT	24
Deutsch	25
Englisch	26
Französisch	27
Spanisch	28
Polnisch	29
Russisch	30
Inhalt	31
Liste der Tabellen	36
Über dieses Handbuch	36
Bezeichnungen	36
Syntax Diagramme	36
Über die Sprache Pascal	37
1. Pascal Zeichen und Symbole	38
2. Konstanten	38
3. Typen	38
4. Variablen	38
5. Objekte	38
6. Klassen	38
7. Schnittstellen (Interface's)	39
8. Generics	39
9. Erweiterte Record's	39
10. Klassen, Record's, und Typen-Helfer	39
11. Objektorientierte Pascal - Klassen	39
12. Ausdrücke	39
13. Anweisungen	39
14. Benutzung von Funktionen und Prozeduren	39
15. Operatoren Überladung	39
16. Programme, Module und Blöcke	39
17. Ausnahmen	39
18. Assembler	39
18.1. Anweisungen	39
18.2. Prozeduren und Funktionen	39

Lizenz

Bitte lesen Sie diese Lizenz gründlich durch, bevor Sie Lazarus die Dokumentation oder Teile dieser Software verwenden wollen. Sollten Sie nicht mit den hier aufgeführten Bedingungen einverstanden sein, ist die Nutzung oder Verwendung des Quellcodes zu diesen Produkt (oder auch Teile davon) nicht gestattet.

Bei der Verwendung darf kein kommerzieller Zweck der Gewinnerzielung entstehen !

Freeware und Shareware-Programme zeichnen sich vor allem dadurch aus, dass sie kostenlos beziehungsweise gegen einen relativ geringen Preis dem Nutzer die Verwendung teilweise hochwertiger Computerprogramme ermöglichen, die sich durchaus mit kommerziellen Produkten messen lassen können. Daher sind diese Softwaretypen gerade in Zeiten begrenzter finanzieller Mittel - auch im schulischen Umfeld sehr beliebt und kommen in vielfältigen Gebieten zum Einsatz (zum Beispiel bei der Netzwerkadministration, bei der Einrichtung von Servern, bei der Installation von Internet-Software auf den Nutzer-PCs oder bei Office-Anwendungen).

Da es sich auch bei Free- und Sharewareprogrammen um Computerprogramme handelt, die gemäß § 69a Urheberrechtsgesetz (UrhG) urheberrechtlich geschützt sind, liegt es bei diesen Softwareprodukten weitgehend in den Händen des Rechteinhabers zum Beispiel des Programmierers - zu bestimmen, in welchem Umfang diese durch dritte Personen genutzt werden dürfen.

Dabei können Freeware- und Sharewareprogramme nach den Lizenzbedingungen in der Regel beliebig kopiert und weitergegeben werden, dagegen ist vor allem eine Veränderung der Programme üblicherweise nicht gestattet. Es ist deshalb eigentlich auch nicht korrekt, wenn im Zusammenhang mit Free- und Shareware immer wieder der Begriff der "Public-Domain-Software" verwendet wird, der frei übersetzt "Software, die im öffentlichen Eigentum steht" bedeutet. Denn dies impliziert, dass die Software von der Öffentlichkeit beliebig genutzt und damit auch verändert werden darf (also gemeinfrei ist); letzteres ist bei Free- und Shareware aber gerade nicht der Fall.

Shareware kann man in zahlreiche Unterkategorien einteilen. So kann Shareware beispielsweise werbegestützte Software oder "Adware" sein, welche dem User Werbung anzeigen soll. Dies hat den Zweck Einnahmen für den Eigentümer zu generieren. Eine weitere Art bezeichnet man als Demoware, wobei es sich hier nur um eine Demoversion der Software handelt.

Dabei ist es oft eine Testversion mit einer festgelegten Zeitspanne ("Trialware"). In anderen Fällen kann die gesamte Funktionalität der Anwendung deaktiviert sein, so dass man zwar alle Funktionen sehen kannst, aber dafür bezahlen muss ("Crippleware").

Weiterhin gibt es auch "Freemium"-Shareware, bei welcher einige Funktionen in der kostenlosen Version verfügbar sind, Sie diese jedoch bezahlen müssen, um die volle Funktionalität freischalten zu können. Obwohl Shareware eine gute Option für jeden ist, der Software testen möchte, bevor man sich zu einem Kauf verpflichtet, sollten Sie dennoch vorsichtig sein. Denn Cyberkriminelle sind nur allzu bereit, den Eifer der Menschen auszunutzen, etwas gratis zu bekommen.

Dieses Versprechen freier Software ist eine gängige Social-Engineering-Taktik, mit der sie Internetnutzer dazu bringen, bösartige Software herunterzuladen.

Bei Shareware gibt es ebenso wie bei anderen Softwareprodukten keine Garantie dafür, dass die Software virenfrei ist. Aus diesem Grund ist es wichtig, dass der Benutzer sein Antivirenprogramm auf dem aktuellen Stand hält, bevor er Shareware herunterlädt.

Prinzipiell kann jeder Shareware herunterladen, solange er einen Computer hat, der die Systemanforderungen des Programms erfüllt. Es gibt jedoch einige Seiten, die nur Benutzern in bestimmten Ländern das Herunterladen von Shareware erlauben.

Nicht alle Shareware-Programme werden überall zum Kauf angeboten. Der beste Weg, um zu sehen, wo man ein bestimmtes Programm kaufen kann, ist, die Website des Entwicklers zu besuchen und herauszufinden, auf welchen Plattformen das Programm erhältlich ist.

Freeware ist Software, die kostenlos heruntergeladen werden kann, aber keine Lizenzgebühren mehr verlangt. Im Gegensatz dazu erfordert Shareware normalerweise eine Lizenzgebühr für den vollen Zugang zu allen Funktionen. Beide Modelle ermöglichen es Benutzern, Software kostenlos auszuprobieren, aber

Shareware ist normalerweise etwas umfangreicher und bietet mehr Funktionen.

Dieses Software-Produkt besitzt eine Hybrid-Lizenz zwischen Free- und Share-Ware. Sie können es kostenlos einsetzen und an andere Personen weitergeben, solange Sie keine Teile der Software ändern oder kopieren. Es bedarf einer schriftlichen Einwilligung des Entwicklers (Jens Kallup), sollten Teile geändert oder übernommen werden.

Sie können dieses Produkt auf mehreren lokalen Computer gleichzeitig nutzen. Eine öffentlich zugängliche Nutzung ist nicht bestrebt mit der aktuellen Version.

Obwohl bei der Entwicklung dieser Software viel Sorgfalt gepflegt wurde, können Fehler nicht ausgeschlossen werden.

Es werden daher keine Garantien auf entstanden Schäden oder Kosten übernommen, die während der Verwendung dieser Software entstehen. Alles erfolgt auf Eigenes Risiko !!!

Für zukünftige Entwicklungen können Sie jedoch einen kleinen Obolus in Form von Spenden hinterlegen.

Überblick

Free Pascal ist ein ausgereifter, vielseitiger, quelloffener Pascal-Compiler. Er kann für viele Prozessorarchitekturen eingesetzt werden: Intel x86 (16 und 32 bit), AMD64/x86-64, PowerPC, PowerPC64, SPARC, SPARC64, ARM, AArch64, MIPS, Motorola 68k, AVR, und die JVM. Zu den unterstützten Betriebssystemen gehören Windows (16/32/64 Bit, CE und natives NT), Linux, Mac OS X/iOS/iPhoneSimulator/Darwin, FreeBSD und andere BSD-Varianten, DOS (16 Bit oder 32 Bit DPMI), OS/2, AIX, Android, Haiku, Nintendo GBA/DS/Wii, AmigaOS, MorphOS, AROS, Atari TOS und verschiedene Embedded-Plattformen. Zusätzlich ist in der Entwicklungsversion Unterstützung für RISC-V (32/64), Xtensa und Z80 Architekturen und für die LLVM Compiler Infrastruktur verfügbar. Zusätzlich betreut das Free Pascal Team einen Transpiler für Pascal nach Javascript namens pas2js.

Aktuelle Version

Version 3.2.2 ist die letzte stabile Version von Free Pascal. Klicken Sie auf den **Download-Link** und wählen Sie einen Spiegel in Ihrer Nähe, um Ihre Kopie herunterzuladen. Die Entwicklungsversionen haben die Versionsnummern 3.3.x. Auf der Entwicklungsseite erfahren Sie, wie Sie die neuesten Quellen erhalten und die Entwicklung unterstützen können.

Funktionen

Die Sprachsyntax ist sehr gut kompatibel mit TP 7.0 sowie mit den meisten Versionen von Delphi (Klassen, rtti, Exceptions, AnsiString's, WideString's, Interfaces). Es gibt auch einen Mac Pascal Modus, der weitgehend kompatibel mit Think Pascal und MetroWerks Pascal ist. Darüber hinaus unterstützt Free Pascal Funktionsüberladung, Operatorüberladung, globale Eigenschaften und einige andere zusätzliche Funktionen.

Erforderlich

x86-Architektur:

Für die 80x86-Version ist mindestens ein 386er Prozessor erforderlich, ein 486er wird jedoch empfohlen.

Die Mac OS X-Version erfordert Mac OS X 10.4 oder höher, wobei die Entwicklerwerkzeuge installiert sein müssen.

PowerPC-Architektur:

Jeder PowerPC-Prozessor ist geeignet. 16 MB RAM sind erforderlich. Es wird erwartet, dass die Mac OS classic Version mit System 7.5.3 und höher funktioniert. Die Mac OS X Version benötigt Mac OS X 10.3 oder später (kann für 10.2.8 oder später kompiliert werden), mit den installierten Entwicklerwerkzeugen. Auf anderen Betriebssystemen läuft Free Pascal auf jedem System, das das Betriebssystem ausführen kann.

ARM-Architektur:

16 MB RAM sind erforderlich. Läuft auf jeder ARM-Linux-Installation.

Sparc-Architektur:

16 MB RAM sind erforderlich. Läuft auf jeder Sparc-Linux-Installation (Solaris ist experimentell).

Lizenz

Die Pakete und die Laufzeitbibliothek stehen unter einer modifizierten Library GNU Public License, um die Verwendung statischer Bibliotheken bei der Erstellung von Anwendungen zu ermöglichen. Der Quellcode des Compilers selbst steht unter der GNU General Public License. Die Quellen sowohl für den Compiler als auch für die Laufzeitbibliothek sind verfügbar; der komplette Compiler ist in Pascal geschrieben.

GNU DOKUMENTATION

GNU Manifest

Free Software und OpenSource Software

Von Richard M. Stallman

Deutsche Übersetzung: Peter Gerwinski

Den offiziellen englischen Originaltext finden Sie unter <http://www.gnu.org/gnu/manifesto.html>.

Anmerkung des Übersetzers: Dies ist eine inoffizielle Übersetzung des GNU-Manifests. Sie soll helfen, den Text zu verstehen und damit zur Verbreitung des GNU-Projekts beitragen. Der Übersetzer übernimmt jedoch keinerlei Garantie für die Richtigkeit der Übersetzung; wenn Sie in offiziellen Angelegenheiten bezug auf das GNU-Manifest nehmen wollen, halten Sie sich bitte an den englischsprachigen Originaltext.

This is an unofficial translation of the GNU Manifesto into German. The translation is intended to help people to understand the text and to help the GNU project to spread. The translator does not provide any warranty for correctness of the translation; if you intend to refer to GNU Manifesto in official concerns, please refer to the authentic English version.

Das unten abgedruckte GNU-Manifest wurde von Richard Stallman zur Entstehungszeit des GNU-Projekts geschrieben, um zu Teilnahme und Unterstützung aufzurufen. Während der ersten Jahre wurde es geringfügig geändert, um neue Entwicklungen zu berücksichtigen, aber jetzt erscheint es uns am besten, es in der Form zu belassen, in der es die meisten Leute gesehen haben.

Seit dieser Zeit haben wir Einiges über häufige Mißverständnisse gelernt, die sich durch eine andere Wortwahl hätten vermeiden lassen. 1993 wurden Fußnoten ergänzt, um diese Punkte zu klären.

Aktuelle Informationen über verfügbare GNU-Software finden Sie auf den GNU-Webseiten, insbesondere in der Software-Liste. Wie Sie selbst beitragen können, finden Sie unter <http://www.gnu.org/help>.

Was ist GNU? GNU ist nicht gleich Unix!

GNU steht für „GNU ist nicht gleich Unix“ und ist der Name eines kompletten, Unix-kompatiblen Software-Systems, das ich gerade schreibe, um es dann an jeden, der es brauchen kann, frei weiterzugeben. (1) Einige andere Freiwillige unterstützen mich. Beiträge in Gestalt von Zeit, Geld, Programmen und Material werden dringend benötigt.

Bis jetzt haben wir einen Emacs-Texteditor mit Lisp, um Editorkommandos zu schreiben, einen Quelltextdebugger, einen yacc-kompatiblen Parsergenerator, einen Linker und etwa 35 Dienstprogramme. Ein Kommandointerpreter (*shell*) ist beinahe fertig. Ein neuer, portabler, optimierender C-Compiler hat sich selbst kompiliert und könnte dieses Jahr herausgegeben werden. Anfänge zu einem *Kernel* existieren, aber es werden noch viele Funktionen benötigt, um Unix zu emulieren. Sobald der Kernel und der Compiler vollendet sind, wird es möglich sein, ein zur Programmentwicklung einsatzfähiges GNU-System herauszugeben. Wir werden TeX als Textsatzsystem einsetzen, aber auch an einem *nroff* wird gearbeitet. Auch werden wir das freie, portable X-Window-System verwenden. Danach werden wir ein portables *Common Lisp* hinzufügen, ein *Empire*-Spiel, ein Tabellenkalkulationsprogramm und hunderte anderer Dinge plus *on-line*-Dokumentation. Wir hoffen, schließlich alles Nützliche, was normalerweise zu einem Unix-System gehört, anbieten zu können – und mehr.

GNU wird in der Lage sein, Unix-Programme laufen zu lassen, aber es wird nicht mit Unix identisch sein. Auf der Grundlage unserer Erfahrungen mit anderen Betriebssystemen werden wir alle gebräuchlichen Verbesserungen vornehmen; insbesondere sind längere Dateinamen geplant, Datei-Versionsnummern, ein absturzsicheres Dateisystem, eventuell Dateinamen-Komplettierung, terminalunabhängige Ausgabe und schließlich ein auf Lisp basierendes Window-System, durch welches mehrere Lisp-Programme und gewöhnliche Unix-Programme ein- und denselben Bildschirm miteinander teilen können. Sowohl C als auch Lisp werden als Systemsprachen verfügbar sein. Für die Kommunikation beabsichtigen wir, UUCP, MIT-Chaosnet sowie die Internet-Protokolle zu unterstützen.

GNU zielt zunächst auf Maschinen der 68000/16000-Klasse mit virtuellem Speicher, weil es auf diesen am leichtesten lauffähig gemacht werden kann. Die Zusatzarbeit, es auf kleinere Maschinen zu portieren, überlassen wir jemandem, der es auf diesen verwenden will.

Um fürchterliche Verwechslungen zu vermeiden, sprechen Sie bitte auch im Englischen das „G“ in „GNU“ mit aus.

Warum ich GNU schreiben muß

Ich glaube, daß es das Gebot der Nächstenliebe verlangt, daß ich ein Programm, das mir gefällt, mit anderen teile, denen es ebenfalls gefällt. Software-Anbieter hingegen wollen die Anwender isolieren und beherrschen, wobei sie jeden Anwender dazu verpflichten, nicht mit anderen zu teilen. Ich weigere mich, die Solidarität mit anderen Anwendern in dieser Weise zu brechen. Ich kann nicht mit gutem Gewissen einen Nichtoffenbarungsvertrag oder einen Software-Lizenzvertrag unterzeichnen.

Damit ich ehrlich bleiben und trotzdem weiterhin Computer benutzen kann, habe ich mich entschlossen, eine genügend große Sammlung von freier Software zusammenzustellen, so daß ich in der Lage sein werde, ohne jegliche nicht-freie Software auszukommen. Ich habe meinen Beruf im *AI lab* aufgegeben, um dem *MIT* keinen rechtlichen Vorwand zu bieten, mich daran zu hindern, GNU weiterzugeben.

Warum GNU Unix-kompatibel sein wird

Unix ist nicht mein ideales Betriebssystem, aber es ist nicht übel. Die wesentlichen Eigenschaften von Unix scheinen gut zu sein, und ich denke, daß ich fehlendes ergänzen kann, ohne die guten Eigenschaften zu verderben. Außerdem wird ein Unix-kompatibles System für viele Menschen eher annehmbar sein.

Wie GNU erhältlich sein wird

GNU ist nicht in der *public domain*. Zwar wird jedem gestattet sein, GNU zu modifizieren und weiterzugeben, aber keinem Distributor wird es erlaubt sein, die Weiterverbreitung von GNU einzuschränken; sprich: proprietäre Modifikationen werden nicht erlaubt sein. Ich möchte sicherstellen, daß alle Versionen von GNU frei bleiben.

Warum viele andere Programmierer mithelfen wollen

Ich habe viele andere Programmierer gefunden, die vom GNU-Projekt begeistert sind und ihre Hilfe anbieten.

Viele Programmierer sind mit der Kommerzialisierung von Systemsoftware unzufrieden. Es mag ihnen die Möglichkeit geben, mehr Geld zu machen, aber es zwingt sie gleichzeitig, andere Programmierer im allgemeinen als Gegner anstatt als Kameraden zu betrachten. Der fundamentale Akt der Freundschaft zwischen Programmierern ist das Teilen von Programmen; derzeitige Vermarktungspraktiken verbieten Programmierern im wesentlichen, sich gegenseitig als Freunde zu behandeln. Der Käufer von Software hat die Wahl zwischen Freundschaft und Gesetzestreue. Naturgemäß entscheiden viele, daß Freundschaft für sie wichtiger ist, aber diejenigen, welche an das Gesetz glauben, haben eine schwere Entscheidung. Sie werden zynisch und betrachten Programmierung nur noch als eine Möglichkeit, Geld zu machen.

Wenn wir an und mit GNU anstelle von proprietären Programmen arbeiten, können wir gleichzeitig zu jedem gastfreundlich sein und dem Gesetz genügen. Außerdem dient GNU uns als inspirierendes Beispiel und als Banner, andere zu sammeln, um sich uns beim Teilen anzuschließen. Dies kann uns ein Gefühl der Harmonie bringen, das beim Gebrauch nicht-freier Software unmöglich wäre. Für jeden zweiten Programmierer, mit dem ich gesprochen habe, ist dies ein wichtiges Glück, das durch Geld nicht ersetzt werden kann.

Wie Sie selbst beitragen können

Ich bitte Computerhersteller um Spenden in Gestalt von Maschinen und Geld. Ich bitte Einzelpersonen um Spenden in Gestalt von Programmen und Arbeit.

Wenn Sie uns eine Maschine zur Verfügung stellen, können Sie damit rechnen, daß GNU relativ früh darauf laufen wird. Die Maschinen sollten komplette, gebrauchsfertige Systeme sein, in einer Wohnung benutzt werden können und keine außergewöhnliche Kühlung oder Energieversorgung benötigen.

Ich habe sehr viele Programmierer gefunden, die bereit sind, einen Teil ihrer Arbeitszeit GNU zu widmen. Für die meisten Projekte würde derartig verteilte Teilzeitarbeit schwer zu koordinieren sein; die voneinander unabhängig geschriebenen Teile würden nicht zusammenarbeiten. Für die spezielle Aufgabe jedoch, Unix zu ersetzen, existiert dieses Problem nicht. Ein komplettes Unix-System enthält hunderte von Dienstprogrammen, von denen jedes separat dokumentiert ist. Die meisten Schnittstellen sind durch Unix-Kompatibilität festgelegt. Wenn jeder Beteiligte einen kompatiblen Ersatz für ein Unix-Dienstprogramm schreibt und dafür sorgt, daß es an der Stelle der Originalkomponente richtig arbeitet, dann werden diese Dienstprogramme auch richtig arbeiten, wenn man sie zusammensetzt. Selbst wenn wir Murphy erlauben, ein paar unerwartete Probleme zu produzieren, sollte das Zusammensetzen dieser Komponenten eine durchführbare Aufgabe sein. (Der Kernel wird eine engere Zusammenarbeit erfordern, daher wird eine kleine Gruppe daran arbeiten.)

Sollte ich Geldspenden erhalten, werden mich diese in die Lage versetzen, ein paar Leute in Voll- oder Teilzeitarbeit einzustellen. Die Gehälter werden für den Standard von Programmierern nicht hoch sein, aber ich suche Leute, für die das Bilden von Gemeinschaftsgeist wichtiger ist als Geld zu scheffeln. Ich sehe dies als einen Weg an, es ausgewählten Leuten zu ermöglichen, ihre gesamte Energie in ihre Arbeit an GNU zu investieren, indem ich sie von der Notwendigkeit freimache, ihren Lebensunterhalt auf andere Weise zu verdienen.

Warum dies allen Computerbenutzern nützen wird

Sobald GNU geschrieben sein wird, wird jeder in der Lage sein, gute Systemsoftware so frei wie Luft zu bekommen. (2)

Dies bedeutet mehr, als nur jedem den Preis für eine Unix-Lizenz zu ersparen. Es bedeutet, daß viele überflüssige Anstrengungen, Systemsoftware jedesmal neu zu programmieren, vermieden werden können. Dieselben Anstrengungen können stattdessen eingesetzt werden, uns auf dem Gebiet weiterzubringen.

Der komplette Quelltext des Systems wird für jeden verfügbar sein mit dem Ergebnis, daß ein Benutzer, der Veränderungen in dem System benötigt, immer die Freiheit haben wird, diese selbst vorzunehmen oder einen Programmierer oder eine Firma damit zu beauftragen. Die Benutzer werden nicht länger von der Gunst eines einzigen Programmierers oder einer einzigen Firma abhängig sein, welche den Quelltext besitzt und daher als einzige Veränderungen vornehmen kann.

Schulen können ein besseres pädagogisches Umfeld bieten, wenn sie die Schüler dazu anhalten, den Code des Betriebssystems zu studieren und zu verbessern. Das *Harvard's computer lab* verlangte, daß kein Programm im System installiert werden durfte, dessen Quelltext nicht öffentlich zugänglich war – und hielten dies aufrecht, indem bestimmte Programme tatsächlich nicht installiert wurden. Dies hat mich sehr inspiriert.

Und schließlich wird auch der Verwaltungsaufwand vermieden, zu überlegen, wem die Systemsoftware gehört und was man damit tun darf und was nicht.

Wenn Menschen für das Benutzen eines Programms einschließlich des Anfertigens von Kopien zahlen müssen, entstehen fürchterliche Kosten für die Gesellschaft durch den schwerfälligen Mechanismus, der notwendig ist, um herauszufinden für wie viel (d.h. für welche Programme) eine Person zahlen muß, und nur ein Polizeistaat kann jeden dazu zwingen, diese Verträge einzuhalten. Stellen Sie sich eine Raumstation vor, in der die Luft zu hohen Kosten hergestellt werden muß: Es mag fair sein, Atemluft pro verbrauchten Liter zu berechnen, aber das Tragen von Gasmasken mit Meßeinrichtungen – Tag und Nacht – ist intolerabel, selbst wenn der Luftpreis für jeden erschwinglich ist. Und allgegenwärtige Fernsehkameras, die überwachen, ob jemand die Maske abnimmt, sind abscheulich. Es ist besser, die Luftanlage durch eine Kopfsteuer zu finanzieren und die Masken wegzuerwerfen.

Das ganze oder teilweise Kopieren eines Programms ist für einen Programmierer so natürlich wie Atmen – und so produktiv. Es sollte genauso frei sein.

Einige leicht zu entkräftende Einwände gegen die Ziele von GNU

„Niemand wird es benutzen, wenn es frei ist, weil dies bedeutet, daß Sie sich nicht auf Wartung verlassen können.“

„Sie müssen etwas für das Programm berechnen, um Service anbieten zu können.“

Wenn die Menschen lieber für GNU mit Service bezahlen, als GNU ohne Service frei zu erhalten, sollte eine Firma, die speziell diesen Service für Leute anbietet, die GNU frei erhalten haben, rentabel sein. (3)

Wir müssen zwischen Wartung in Gestalt von echter Programmierarbeit und Benutzerhilfe unterscheiden. Ersteres ist etwas, das Sie von einem Software-Händler nicht erwarten können: Wenn Ihr Problem nicht von genügend Leuten geteilt wird, wird der Händler Sie zum Teufel schicken.

Wenn Ihr Unternehmen darauf angewiesen ist, sich auf Wartung zu verlassen, ist der einzig gangbare Weg, alle nötigen Quelltexte und Werkzeuge vorliegen zu haben, denn dann können Sie jede verfügbare Person einstellen, Ihr Problem zu lösen, und sind nicht von der Gunst einer speziellen Person abhängig. Mit Unix ist dies infolge des hohen Preises der Quelltexte für die meisten Unternehmen unerschwinglich; mit GNU wird dies leicht sein. Es kann immer noch sein, daß keine kompetente Person zur Verfügung steht, aber dies liegt dann nicht an den Vertriebsbedingungen. GNU löst nicht alle Probleme der Welt, sondern nur bestimmte.

Gleichzeitig sind Anwender ohne Computerwissen auf Hilfe angewiesen: Dinge für sie erledigen, die sie leicht selbst tun könnten, aber nicht wissen, wie.

Derartige Dienste könnten von Firmen angeboten werden, die gerade solche Benutzerhilfen und Reparaturservice anbieten. Wenn es stimmt, daß Benutzer es vorziehen, für ein Produkt mit Service zu bezahlen, werden sie auch bereit sein, den Service zu bezahlen, während sie das Produkt frei erhalten haben. Die Serviceunternehmen werden in Qualität und Preis miteinander konkurrieren; die Benutzer werden nicht an ein spezielles gebunden sein. Gleichzeitig können diejenigen von uns, die den Service nicht benötigen, das Programm benutzen, ohne für den Service bezahlen zu müssen.

„Ohne Werbung können Sie nicht viele Leute erreichen, und Sie müssen etwas für das Programm berechnen, um dies zu ermöglichen.“

„Es ist nutzlos, für etwas zu werben, was man umsonst bekommen kann.“

Es gibt viele Formen kostenloser oder kostengünstiger Werbung, die dazu dienen kann, viele Computerbenutzer über so etwas wie GNU zu informieren. Es mag stimmen, daß man mehr Benutzer von Microcomputern durch Werbung erreichen kann; wenn dies so ist, sollte ein Unternehmen, das etwas für den Service des Kopierens und Verteilens von GNU berechnet, erfolgreich genug sein, um sich Werbung und mehr leisten zu können. Auf diese Weise zahlen auch nur diejenigen Benutzer für die Werbung, die von ihr profitieren.

Wenn andererseits viele Leute GNU von ihren Freunden erhalten und solche Unternehmen keinen Erfolg haben, zeigt dies, daß Werbung in Wirklichkeit gar nicht nötig war, um GNU zu verbreiten. Warum wollen die Vertreter der freien Marktwirtschaft nicht den freien Markt darüber entscheiden lassen? (4)

„Mein Unternehmen benötigt ein proprietäres Betriebssystem, um einen Wettbewerbsvorteil zu bekommen.“

GNU wird Betriebssystem-Software aus dem Bereich des Wettbewerbs entfernen. Sie werden keinen Vorteil auf diesem Gebiet erzielen können, aber umgekehrt wird auch Ihre Konkurrenz Sie nicht übervorteilen können. Sie werden auf anderen Gebieten in Wettbewerb treten, während Sie auf diesem Gebiet voneinander profitieren werden. Wenn Ihr Unternehmen vom Verkauf eines Betriebssystems lebt, werden Sie GNU nicht mögen, aber das ist Ihr Problem. Wenn Ihr Unternehmen anders ist, kann GNU Sie davor bewahren, in das teure Geschäft gedrängt zu werden, Betriebssysteme zu verkaufen.

Ich würde es gerne sehen, wenn viele Hersteller und Benutzer die Entwicklung von GNU durch Spenden unterstützen würden, um die Kosten für jeden einzelnen zu senken. (5)

„Verdienen nicht die Programmierer eine Belohnung für ihre Kreativität?“

Wenn irgendetwas eine Belohnung verdient, dann sind es soziale Beiträge. Kreativität kann ein sozialer Beitrag sein, aber nur, wenn die Gesellschaft die Freiheit hat, die Resultate zu nutzen. Wenn Programmierer eine Belohnung für das Schreiben innovativer Programme verdienen, müßten sie aus demselben Grunde bestraft werden, wenn sie die Nutzung dieser Programme einschränken.

„Sollte ein Programmierer nicht eine Belohnung für seine Kreativität verlangen dürfen?“

Es ist nichts Falsches darin, Bezahlung für Arbeit zu verlangen oder sein Einkommen maximieren zu wollen, solange man nicht destruktiv wird. Die zur Zeit auf diesem Gebiet gebräuchlichen Mittel basieren auf einer Form von Zerstörung.

Geld von Benutzern zu kassieren, indem man den Gebrauch eines Programms einschränkt, ist destruktiv, weil die Einschränkungen die Häufigkeit und die verschiedenen Weisen begrenzen, in denen das Programm benutzt werden könnte. Dies begrenzt den Reichtum, der aus dem Programm für die Menschheit entsteht. Die schädlichen Auswirkungen einer bewußten Beschränkung sind eine bewußte Form von Zerstörung.

Der Grund, weshalb ein guter Bürger derart destruktive Mittel nicht anwendet, um reich zu werden, ist, daß, wenn dies jeder täte, wir alle durch die wechselseitige Zerstörung ärmer würden. Dies ist Kantsche Ethik – oder das Gebot der Nächstenliebe. Die Konsequenzen, die daraus entstünden, daß ein jeder Information horten würde, gefallen mir nicht, daher sehe ich mich gezwungen, es für falsch zu befinden, wenn sich einer so verhält. Insbesondere begründet der Wunsch, für Kreativität belohnt zu werden, es nicht, die gesamte Welt all dieser Kreativität oder von Teilen davon zu berauben.

„Werden die Programmierer nicht verhungern?“

Ich könnte antworten, daß niemand gezwungen ist, ein Programmierer zu sein. Die meisten von uns könnten nicht davon leben, auf der Straße zu stehen und Possen zu reißen, aber wir sind deswegen noch lange nicht dazu verdammt, auf der Straße zu stehen, Possen zu reißen und zu verhungern – wir machen etwas anderes.

Aber dies ist die falsche Antwort, weil sie die implizite Annahme des Fragestellers akzeptiert, daß nämlich Programmierer keinen Pfennig erhalten würden, wenn es keinen Besitz von Software gibt. Es wird angenommen, daß es um „alles oder nichts“ geht.

Der wirkliche Grund, weshalb Programmierer nicht verhungern werden, ist, daß es für sie immer noch möglich sein wird, Geld für Programmierung zu erhalten, nur eben nicht so viel, wie dies im Moment der Fall ist.

Eingeschränktes Kopieren ist nicht die einzige geschäftliche Basis in Sachen Software. Es ist die üblichste Basis, weil sie am meisten Geld einbringt. Wäre diese Basis verboten oder würde sie von den Kunden abgelehnt, würde sich das Software-Geschäft auf andere organisatorische Grundlagen begeben, die zur Zeit weniger häufig verwendet werden. Es gibt immer viele Wege, Geschäfte zu organisieren.

Sicherlich wird das Programmieren auf dieser neuen Basis nicht so lukrativ sein, wie es jetzt ist, aber dies ist kein Argument gegen den Wechsel. Man betrachtet es im allgemeinen nicht als ungerecht, daß Verkäufer die Gehälter bekommen, die sie bekommen. Würden Programmierer die gleichen Gehälter beziehen, wäre dies ebenfalls nicht ungerecht. (In der Praxis werden sie auch weiterhin deutlich mehr beziehen.)

„Haben Menschen nicht das Recht, zu kontrollieren, wie ihre Ideen verwendet werden?“

„Kontrolle über den Gebrauch von Ideen“ konstituiert in Wirklichkeit Kontrolle über das Leben anderer Menschen, und sie wird normalerweise eingesetzt, um den Menschen das Leben schwerer zu machen.

Leute, die das Thema geistigen Eigentums sorgfältig studiert haben (z.B. Anwälte) sagen, daß es kein intrinsisches Recht auf intellektuelles Eigentum gibt. Die von der Regierung anerkannten Arten angenommenen intellektuellen Eigentums wurden durch spezielle Gesetze für spezielle Zwecke geschaffen.

Beispielsweise wurde das Patentsystem etabliert, um Erfinder zu ermutigen, die Details ihrer Erfindungen zu offenbaren. Der Sinn war eher der Gesellschaft zu helfen als dem Erfinder. Zu jener Zeit war die Lebensdauer von 17 Jahren für ein Patent klein verglichen mit der Geschwindigkeit des Fortschritts. Weil Patente nur für Hersteller ein Thema sind, für welche die Kosten und Mühen einer Lizenz klein verglichen mit den Produktionskosten sind, schaden Patente meistens nicht viel. Die meisten Einzelpersonen, die patentierte Produkte benutzen, werden dadurch nicht behindert.

Die Idee des Urheberrechts existierte früher nicht, als Autoren häufig andere Autoren in nicht-fiktionalen Werken kopierten. Diese Praxis war nützlich, und nur auf diesem Wege sind die Werke vieler Autoren

wenigstens teilweise erhalten geblieben. Das System des Urheberrechts wurde ausdrücklich dafür entwickelt, Autorschaft zu ermutigen. Auf dem Gebiet, für das es erfunden wurde – Bücher, die nur auf einer Druckerpresse auf ökonomische Weise vervielfältigt werden konnten – schadete es wenig, und die meisten Leser wurden dadurch nicht behindert.

Alle intellektuellen Eigentumsrechte wurden von der Gesellschaft lizenziert, weil man – richtig oder falsch – glaubte, daß die gesamte Gesellschaft von der Einführung dieser Rechte profitieren würde. In einer speziellen Situation jedoch müssen wir uns fragen, ob wir wirklich besser damit fahren, solche Rechte zu lizenzieren. Welche Art von Handlungen erlauben wir den Personen?

Der Fall von Computerprogrammen heute ist sehr verschieden von dem von Büchern vor hundert Jahren. Die Tatsache, daß der einfachste Weg, ein Programm zu kopieren, von Nachbar zu Nachbar geht, die Tatsache, daß zu einem Programm sowohl Quelltext als auch Objectcode gehören, die verschieden sind, und die Tatsache, daß ein Programm benutzt und nicht gelesen und genossen wird, schaffen zusammen eine Situation, in der eine Person, die ein Urheberrecht einfordert, der gesamten Gesellschaft sowohl materiellen als auch spirituellen Schaden zufügt – eine Situation, in der eine Person kein Urheberrecht einfordern sollte, unabhängig davon, ob die Gesetze es ihr erlauben.

„Im Wettbewerb werden Dinge besser ausgeführt.“

Das Musterbeispiel eines Wettbewerbs ist ein Wettrennen: Indem wir den Gewinner belohnen, ermutigen wir jeden Teilnehmer, schneller zu laufen. Wenn der Kapitalismus tatsächlich auf diese Weise funktioniert, ist es gut, aber seine Befürworter haben Unrecht mit der Annahme, daß es immer so funktioniert. Wenn die Läufer vergessen, weshalb der Preis ausgesetzt wurde und unbedingt gewinnen wollen – egal wie –, entdecken sie vielleicht andere Strategien, zum Beispiel, andere Läufer anzugreifen. Wenn die Läufer in einen Faustkampf geraten, werden alle erst spät durchs Ziel gehen.

Proprietäre und geheime Software sind das moralische Äquivalent von Rennläufern in einem Faustkampf. Es ist traurig, daß sich der einzige vorhandene Schiedsrichter nicht um Faustkämpfe sorgt und sie lediglich reguliert („Pro zehn gelaufene Meter dürft ihr einen Schuß abfeuern“). Er sollte sie stattdessen auseinanderbringen und Läufer bereits für den Versuch eines Angriffs bestrafen.

„Wird ohne finanziellen Ansporn nicht jeder aufhören zu programmieren?“

Tatsächlich werden viele Menschen absolut ohne jeden finanziellen Ansporn programmieren. Das Programmieren übt eine unwiderstehliche Faszination auf manche Leute aus – normalerweise diejenigen Leute, die darin am besten sind. Es gibt keinen Mangel an Berufsmusikern, die bei ihrem Beruf bleiben, obwohl sie keinerlei Hoffnung haben, damit ihren Lebensunterhalt bestreiten zu können.

Aber in Wirklichkeit ist diese Frage, obwohl sie häufig gestellt wird, der Situation nicht angemessen. Die Bezahlung für Programmierer wird nicht verschwinden, sondern nur weniger werden. Die richtige Frage ist daher: Wird irgendjemand bei reduziertem finanziellen Ansporn programmieren? Meine Erfahrung zeigt, daß jemand es tun wird.

Vor über zehn Jahren haben einige der besten Programmierer der Welt im *Artificial Intelligence Lab* für weit weniger Geld gearbeitet, als sie anderswo hätten verdienen können. Sie erhielten nicht-finanzielle Belohnungen in vielerlei Art, zum Beispiel Berühmtheit und Dank. Außerdem ist die Kreativität selbst eine Freude, eine Belohnung für sich.

Dann geschah es, daß die meisten dieser Programmierer verschwanden, als sie eine Chance bekamen, dieselbe interessante Arbeit für viel Geld zu tun.

Die Tatsachen zeigen, daß Menschen aus anderen Gründen als Bereicherung programmieren werden, aber wenn man ihnen eine Möglichkeit gibt, außerdem noch viel Geld zu scheffeln, geschieht es, daß sie dieses Geld erwarten und verlangen. Niedriglohn-Organisationen machen sich schlecht neben Höchstlohn-Organisationen; dies entfällt, wenn die Höchstlohn-Organisationen verboten sind.

„Wir sind auf die Programmierer angewiesen. Wenn die verlangen, daß wir aufhören, unseren Nachbarn zu helfen, müssen wir nachgeben.“

Sie können niemals so verzweifelt sein, daß Sie derartigen Forderungen nachgeben müssen. Vergessen Sie

nicht: Millionen für die Verteidigung – keinen Pfennig für Tribut!

„Programmierer müssen von irgendetwas leben.“

Kurzfristig existiert dieses Problem. Es gibt aber viele Wege, wie Programmierer ihren Lebensunterhalt bestreiten können, ohne das Recht zu verkaufen, eine Programm zu benutzen. Der derzeitige Weg ist üblich, weil er Programmierern und Geschäftsleuten das meiste Geld einbringt und nicht, weil es der einzige Weg ist, seinen Lebensunterhalt zu verdienen. Es ist leicht, andere Wege zu finden, wenn man sie sucht; einige Beispiele folgen.

Ein Computerhersteller, der einen neuen Computer einführt, zahlt für die Portierung des Betriebssystems auf die neue Hardware.

Programmierer können in der Schulung, Benutzerhilfe und Wartung unterkommen.

Leute mit neuen Ideen können Programme als FreeWare verteilen und zufriedene Benutzer um Spenden bitten oder Benutzerhilfen anbieten. Ich bin einigen Leuten begegnet, die bereits erfolgreich in dieser Weise arbeiten.

Benutzer mit ähnlichen Bedürfnissen können Interessensgemeinschaften bilden und Beiträge zahlen. Die Gruppe würde dann Programmierfirmen damit beauftragen, Programme zu schreiben, die die Mitglieder gerne benutzen würden.

Alle Arten von Weiterentwicklung könnten durch eine Software-Steuer finanziert werden:

Stellen wir uns vor, jeder, der einen Computer kauft, muß x Prozent des Preises an Software-Steuer entrichten. Die Regierung gibt dieses Geld einer Agentur wie der NSF, um es für Software-Entwicklung einzusetzen.

Wenn allerdings der Computerkäufer selbst für Software-Entwicklung spendet, wird die Spende mit der Software-Steuer verrechnet. Er kann für das Projekt seiner Wahl spenden – häufig in der Hoffnung ausgewählt, das Ergebnis benutzen zu können. Er kann beliebig hohe Spenden anrechnen lassen bis hin zum Gesamtvolumen der zu zahlenden Steuer.

Der Steuersatz könnte durch die Steuerzahler selbst durch Wahl festgelegt werden, gewichtet gemäß der Menge der zu zahlenden Steuer.

Die Konsequenzen:

- *Die Gemeinschaft der Computerbenutzer unterstützt die Software-Entwicklung.*
- *Die Gemeinschaft entscheidet, wieviel Unterstützung benötigt wird.*
- *Benutzer, denen es darauf ankommt, an welchen Projekten sie sich beteiligen, können dies selbst entscheiden.*

Auf lange Sicht ist das Freigeben von Programmen ein Schritt in Richtung einer Welt ohne Mangel, in der niemand hart arbeiten muß, um sein Leben zu bestreiten. Die Menschen werden frei sein, sich Aktivitäten zu widmen, die Freude machen, zum Beispiel Programmieren, nachdem sie zehn Stunden pro Woche mit notwendigen Aufgaben wie Verwaltung, Familienberatung, Reparatur von Robotern und der Beobachtung von Asteroiden verbracht haben. Es wird keine Notwendigkeit geben, von Programmierung zu leben.

Wir haben bereits die Menge an Arbeit, welche die Gesellschaft für ihre Produktivität aufbringen muß, gewaltig reduzieren können, aber nur ein kleiner Teil davon übertrug sich in mehr Freizeit für Arbeiter, weil jede produktive Aktivität zwangsläufig von viel unproduktiver Aktivität begleitet wird. Die Hauptursache hierfür ist Bürokratie und gegenseitiges Bekämpfen, ohne dabei voran zu kommen, anstelle von Wettbewerb. Freie Software wird diese Auswüchse auf dem Gebiet der Software-Entwicklung in großartiger Weise reduzieren. Wir müssen so handeln, um technische Fortschritte in Sachen Produktivität zu erzielen, die sich in weniger Arbeit für uns alle äußern werden.

Fußnoten:

(1)

Diese Wortwahl war ein wenig sorglos. Die Absicht war, daß niemand für die *Erlaubnis* zahlen muß, das GNU-System zu benutzen. Dies geht jedoch nicht aus der Formulierung hervor, und man interpretiert dies häufig so, daß Kopien von GNU stets gegen kein oder höchstens geringes Entgelt verteilt werden sollen. Dies war niemals die Absicht; weiter unten erwähnt das Manifest die Möglichkeit für Firmen, GNU gegen Bezahlung zu verteilen. Später habe ich es gelernt, sorgfältig zwischen „frei“ im Sinne von „Freiheit“ und „frei“ im Sinne von „Preis“ zu unterscheiden. Freie Software ist Software, deren Benutzer die Freiheit haben, sie weiterzugeben und zu verändern. Manche werden ihre Kopien gratis erhalten, während andere dafür bezahlen werden – und wenn diese Geldsumme dazu beiträgt, die Software weiter zu verbessern, um so besser. Wichtig ist, daß jeder, der eine Kopie hat, auch die Freiheit hat, beim Gebrauch dieser Kopie mit anderen zu kooperieren.

(2)

Dies ist eine weitere Stelle, an der ich versämut habe, sorgfältig zwischen den beiden verschiedenen Bedeutungen von „frei“ zu unterscheiden. So, wie der Satz geschrieben ist, ist er nicht falsch – Sie können Kopien von GNU-Software kostenlos von Ihren Freunden oder über das Netz erhalten –, aber er suggeriert die falsche Idee.

(3)

Inzwischen gibt es mehrere solche Firmen.

(4)

Die *Free Software Foundation* bezieht den größten Teil ihres Kapitals von einem solchen Verteilungsservice, obwohl sie eher eine caritative Einrichtung als eine Firma ist. Wenn sich *niemand* dafür entscheidet, seine Kopien von der FSF zu beziehen, wird diese ihre Arbeit nicht machen können. Dies bedeutet nicht, daß proprietäre Einschränkungen gemacht werden sollten, um jeden Benutzer zum Bezahlen zu zwingen; wenn ein kleiner Bruchteil aller Benutzer seine Kopien von der FSF bezieht, genügt dies, die FSF flüssig zu halten. Daher bitten wir Benutzer in dieser Weise um ihre Unterstützung. Haben Sie Ihren Teil bereits geleistet?

(5)

Eine Gruppe von Computerunternehmen hat vor kurzem Kapital angesammelt, um die Wartung des GNU-C-Compilers zu unterstützen.

* * *

Copyright-Notiz des englischsprachigen Originals:

Copyright © 1985, 1993, 2003, 2005 Free Software Foundation, Free Software Foundation, Inc., 51 Franklin St, Fifth Floor, Boston, MA 02110-1301, USA

Permission is granted to anyone to make or distribute verbatim copies of this document, in any medium, provided that the copyright notice and permission notice are preserved, and that the distributor grants the recipient permission for further redistribution as permitted by this notice.

Modified versions may not be made.

Übersetzung:

Es ist jedem gestattet, unveränderte Kopien dieses Dokuments in beliebigen Medien anzufertigen oder weiterzugeben, vorausgesetzt, daß diese Copyright-Notiz und diese Erlaubnis erhalten bleiben und daß der Distributor den Empfängern das Recht zur Weiterverbreitung im Sinne dieser Erlaubnis gewährt.

Veränderte Versionen dürfen nicht erstellt werden.

Weshalb „freie Software“ besser ist als „Open Source“

Von **Richard M. Stallman**

Deutsche Übersetzung: Peter Gerwinski, Dezember 2006

Den offiziellen englischen Originaltext finden Sie unter <http://www.gnu.org/philosophy/free-software-for-freedom.html>. Dieser Artikel wurde außerdem in dem Buch Free Software, Free Society: The Selected Essays of Richard M. Stallman veröffentlicht.

Freie Software unter irgendeinem anderen Namen würde Ihnen dieselbe Freiheit geben. Trotzdem ist es ein großer Unterschied, welchen Namen Sie verwenden: Verschiedene Worttransportieren verschiedene Konzepte.

1998 begannen einige Leute in der Gemeinschaft der freien Software, den Begriff „Open-Source-Software“ anstelle von „freie Software“ zu verwenden, um ihre Tätigkeiten zu beschreiben. Schon bald wurde der Begriff „Open Source“ mit einem anderen Ansatz, einer anderen Philosophie und anderen Werten assoziiert und sogar mit unterschiedlichen Kriterien dafür, welche Software-Lizenzen akzeptabel seien. Heutzutage handelt es sich bei der Freie-Software- und der Open-Source-Bewegung um separate Bewegungen mit unterschiedlichen Ansichten und Zielen. Nichtsdestoweniger können wir in der Praxis an einigen Projekten zusammenarbeiten – und tun dies auch.

Der grundsätzliche Unterschied zwischen beiden Bewegungen liegt in ihren Wertesystemen, ihrer unterschiedlichen Art, die Welt zu betrachten. Für die Open-Source-Bewegung ist die Frage, ob Software Open-Source-Software sein soll, eine Frage der Zweckmäßigkeit und nicht der Ethik. Jemand brachte das einmal folgendermaßen auf den Punkt: „Open-Source ist eine Methode zur Software-Entwicklung; freie Software ist eine soziale Bewegung.“ Für die Open-Source-Bewegung ist nicht-freie Software eine suboptimale Lösung. Für die Freie-Software-Bewegung ist nicht-freie Software ein soziales Problem, und freie Software ist die Lösung.

Das Verhältnis zwischen der Freie-Software- und der Open-Source-Bewegung

Die Freie-Software-Bewegung und die Open-Source-Bewegung sind wie zwei politische Parteien innerhalb der Gemeinschaft der freien Software.

Die radikalen Gruppen in den 1960er Jahren waren für ihren Parteigeist berüchtigt: Organisationen fielen wegen Uneinigkeit über Details in der Vorgehensweise auseinander und betrachteten sich von da an als Gegner. Dies ist zumindest das Bild, das man von ihnen hat, ob nun wahr oder falsch.

Das Verhältnis zwischen der Freie-Software- und der Open-Source-Bewegung ist gerade das Gegenteil dieses Bildes. In den Grundprinzipien sind wir uns uneinig, aber hinsichtlich der praktischen Empfehlungen stimmen wir weitgehend überein. Daher können wir an vielen spezifischen Projekten gemeinsam arbeiten und tun dies auch tatsächlich. Wir sehen die Open-Source-Bewegung nicht als Gegner. Der Gegner ist die proprietäre Software.

Wir sind nicht gegen die Open-Source-Bewegung, aber wir wollen nicht mit ihr in einen Topf geworfen werden. Wir erkennen ihre Leistungen für unsere Gemeinschaft an, aber wir waren es, die diese Gemeinschaft aufgebaut haben, und wir möchten, daß die Leute das wissen. Wir wollen, daß man das, was wir erreicht haben, mit unseren Werten und unserer Philosophie in Verbindung bringt, nicht mit den ihrigen. Wir möchten gehört werden und nicht hinter einer Gruppe mit ganz anderen Ansichten versteckt werden. Um zu vermeiden, daß man uns für einen Teil der Open-Source-Bewegung hält, vermeiden wir mühsam das Wort „open“ (offen), wenn wir von freier Software sprechen, und das Wort „closed“ (geschlossen), wenn wir von nicht-freier Software sprechen.

Bitte erwähnen Sie daher die Freie-Software-Bewegung, wenn Sie über unsere Arbeit sprechen und über die Software, die wir entwickelt haben – wie zum Beispiel das Betriebssystem GNU/Linux.

Vergleich beider Begriffe

Der Rest dieses Artikels vergleicht die beiden Begriffe „freie Software“ und „Open Source“ miteinander und zeigt, weshalb der Begriff „Open Source“ keinerlei Probleme löst, sondern im Gegenteil Probleme aufwirft.

Mehrdeutigkeit

Der Begriff „freie Software“ hat ein Mehrdeutigkeitsproblem: Die unbeabsichtigte Bedeutung „Software, die man kostenlos bekommen kann“ paßt auf den Begriff genauso gut wie die beabsichtigte Bedeutung, „Software, die einem gewisse Freiheiten gibt“. Wir gehen dieses Problem dadurch an, daß wir eine präzisere Definition von freier Software bereitstellen, aber diese Lösung ist nicht perfekt und kann das Problem nicht gänzlich beseitigen. Ein eindeutig korrekter Begriff wäre besser, wenn er keine anderen Probleme aufwerfen würde.

Leider sind alle Alternativen im Englischen auf ihre eigene Weise problematisch. Wir haben uns viele Alternativen angesehen, die uns vorgeschlagen wurden, aber keine ist derart eindeutig „richtig“ daß es eine

gute Idee wäre, zu ihr zu wechseln. Jeder vorgeschlagene Ersatz für „freie Software“ wirft ähnlich geartete semantische Probleme auf – oder schlimmere. Dies gilt auch für „Open-Source-Software“.

Die offizielle Definition von „Open-Source-Software“ der Open-Source-Initiative ist unserer Definition von freier Software sehr ähnlich. Nichtsdestoweniger ist sie in mancher Hinsicht eine Spur lockerer, und es wurden ein paar Software-Lizenzen akzeptiert, die wir als für den Benutzer unakzeptabel restriktiv ansehen.

Wie dem auch sei, lautet die offensichtliche Bedeutung des Begriffs „Open-Source“ (quelloffen): „Sie haben Einblick in den Quelltext“. Dies ist ein wesentlich schwächeres Kriterium als freie Software, das zwar freie Software mit einschließt, aber auch semi-freie Computerprogramme wie z.B. Xv und sogar proprietäre Software, darunter Qt unter seiner ursprünglichen Lizenz (vor der QPL).

Diese offensichtliche Bedeutung von „Open Source“ ist nicht die von ihren Befürwortern beabsichtigte. Das Ergebnis ist, daß die meisten Leute mißverstehen, was hier befürwortet wird. Der Autor Neal Stephenson beispielsweise definiert „Open Source“ folgendermaßen:

Linux ist „Open-Source-Software“, was schlicht und einfach bedeutet, daß jedermann seinen Quelltext erhalten kann.

Ich glaube nicht, daß es in seiner Absicht lag, die „offizielle“ Definition zu diskutieren oder in Frage zu stellen. Ich glaube vielmehr, daß er einfach die Regeln der englischen Sprache angewendet hat, um die Bedeutung des Begriffs zu erschließen. Der US-Bundesstaat Kansas publizierte eine ähnliche Definition:

Verwenden Sie Open-Source-Software (OSS). OSS ist Software, deren Quelltext frei und öffentlich zugänglich ist, wenn auch die spezifischen Software-Lizenzen variieren, die besagen, was genau man mit dem Quelltext tun darf.

Natürlich haben die Open-Source-Leute versucht, diesem Problem durch eine präzise Definition des Begriffs zu begegnen, genauso wie wir es für den Begriff „freie Software“ getan haben.

Aber die Erklärungen für „freie Software“ sind einfach: Jemand, der das Konzept „freie Rede, nicht Freibier“ begriffen hat, wird es nicht wieder falsch verstehen. Es gibt keinen derart prägnanten Weg, die offizielle Bedeutung von „Open Source“ zu erklären und klar aufzuzeigen, weshalb die [sprachlich-]natürliche Definition falsch ist.

Angst vor der Freiheit

Das Hauptargument für den Begriff „Open-Source-Software“ ist, daß „freie Software“ manche Leute verängstigt. Es stimmt: Wer über die Freiheit, über ethische Fragen, über Luxus und Verantwortung spricht, fordert die Menschen auf, über Dinge nachzudenken, die sie lieber ignorieren würden. Dies kann Unbehagen auslösen, und manche Leute werden das Konzept deswegen ablehnen. Daraus folgt aber nicht, daß es der Gesellschaft besser erginge, wenn wir über diese Themen nicht mehr reden würden.

Vor Jahren haben die Entwickler freier Software diese Reaktion des Unbehagens erkannt, und manche haben einen Ansatz erkundet, dies zu vermeiden. Sie fanden heraus, daß sie, wenn sie zu Ethik und Freiheit schwiegen und nur über die unmittelbaren praktischen Vorteile bestimmter freier Software sprachen, die Software an gewisse Anwender besser „verkaufen“ konnten, insbesondere an Geschäftskunden. Der Begriff „Open Source“ wird als ein Weg angeboten, der weiter in diese Richtung führt – ein Weg, für Geschäftskunden „akzeptabler“ zu sein. Die Anschauungen und Wertvorstellungen der Open-Source-Bewegung gehen auf diese Entscheidung zurück.

Dieser Ansatz hat sich bewährt – gemäß seinen eigenen Begriffen. Viele Menschen wechseln heutzutage aus rein praktischen Erwägungen heraus zu freier Software. Dies ist, für sich genommen, eine gute Sache, aber damit ist nicht alles getan! Das Gewinnen von Anwendern freier Software ist nicht die vollständige Aufgabe, sondern nur der erste Schritt.

Früher oder später wird man diese Anwender dazu einladen, wegen gewisser praktischer Vorteile zu proprietärer Software zurückzuwechseln. Zahllose Unternehmen streben danach, solche Versuchungen anzubieten. Warum sollten die Anwender sie zurückweisen? Nur deswegen, weil sie gelernt haben, *die Freiheit wertzuschätzen*, die ihnen freie Software gibt, um der Freiheit selbst willen. Es liegt an uns, dieses Konzept zu verbreiten, – und um das zu tun, müssen wir über Freiheit reden. Ein gewisses Maß des

„Schweigen ist Gold“-Ansatzes kann für die Gemeinschaft nützlich sein, aber wir brauchen darüberhinaus reichlich Reden über Freiheit.

Zur Zeit haben wir reichlich Schweigen, aber nicht genug Reden über Freiheit. Die meisten Leute, die mit freier Software zu tun haben, sprechen wenig über Freiheit – typischerweise, weil sie „akzeptabler für Geschäftskunden“ sein wollen. Insbesondere Software-Distributoren zeigen solche Verhaltensweisen. Manche Distributionen des GNU/Linux-Betriebssystems fügen dem freien Basis-System proprietäre Software-Pakete hinzu und fordern die Anwender auf, darin einen Vorteil zu sehen – anstelle eines Rückschritts weg von der Freiheit.

Es gelingt uns nicht, mit dem Zustrom von Anwendern freier Software mitzuhalten. Sie kommen so schnell, daß wir es nicht schaffen, sie über die Freiheit und unsere Gemeinschaft zu unterrichten. Deshalb findet nicht-freie Software (was Qt war, als es populär wurde) und teilweise unfreie Betriebssystem-Distributionen derart fruchtbaren Boden. Jetzt damit aufzuhören, das Wort „frei“ zu verwenden, wäre ein Fehler. Wir brauchen mehr, nicht weniger Reden über Freiheit.

Wenn diejenigen, die den Begriff „Open Source“ benutzen, mehr Anwender in unsere Gemeinschaft holen, ist dies ein Beitrag, aber der Rest von uns muß dann um so härter daran arbeiten, diese Anwendern auf das Konzept der Freiheit aufmerksam zu machen. Wir müssen den Satz „Es ist freie Software, und sie gibt dir Freiheit!“ häufiger und lauter sagen als je zuvor.

Würde eine Marke helfen?

Die Befürworter von „Open-Source-Software“ versuchten, daraus eine Marke zu machen, und sagten, dies würde sie in die Lage versetzen, Mißbrauch zu verhindern. Diese Initiative wurde später fallengelassen; der Begriff sei zu sehr beschreibend, um sich als Marke zu eignen. Daher ist der rechtliche Status von „Open Source“ derselbe wie der von „freie Software“: Es gibt *keinerrechtlichen* Beschränkungen für seinen Gebrauch. Ich habe Berichte von etlichen Fällen gehört, in denen Unternehmen Software-Pakete als „Open Source“ bezeichnet haben, obwohl sie nicht der offiziellen Definition genügten. Einige solche Fälle habe ich selbst beobachtet.

Aber würde es einen großen Unterschied ausmachen, einen Begriff zu verwenden, der eine Marke ist? Nicht notwendigerweise.

Unternehmen haben Ankündigungen herausgegeben, die den Eindruck erweckten, ein Computerprogramm sei „Open-Source-Software“, ohne dies ausdrücklich zu behaupten. Zum Beispiel lautete eine Ankündigung von IBM zu einem Programm, das nicht der offiziellen Definition genügte, folgendermaßen:

Wie in der Open-Source-Gemeinschaft üblich, werden Anwender der ...-Technologie auch mit IBM kooperieren können ...

Hier wird nicht tatsächlich behauptet, die Software *sei* „Open Source“, aber dieses Detail nahmen viele Leser nicht zur Kenntnis. (Ich sollte erwähnen, daß IBM aufrichtig versucht hat, dieses Programm zu freier Software zu machen, und daß IBM später eine neue Lizenz eingeführt hat, die daraus freie Software und „Open Source“ macht, aber als die Ankündigung herauskam, qualifizierte sich das Programm als keins von beidem.)

Und hier sehen wir, wie die Firma Cygnus Solutions, die als Firma für freie Software gegründet worden war und später zu proprietärer Software (um es so auszudrücken) verzweigte, ein proprietäres Software-Produkt bewarb:

Cygnus Solutions ist auf dem Open-Source-Markt ein führendes Unternehmen und hat soeben zwei neue Produkte in den [GNU/Linux]-Markt eingeführt.

Im Gegensatz zu IBM hat Cygnus nicht versucht, diese Software-Pakete zu freier Software zu machen; sie waren weit davon entfernt, sich zu qualifizieren. Aber Cygnus hat nicht wirklich behauptet, es handele sich um „Open-Source-Software“; man hat nur den Begriff benutzt, um bei unachtsamen Lesern diesen Eindruck zu erwecken.

Diese Beobachtungen legen nahe, daß eine Marke die Verwirrung, die mit dem Begriff „Open Source“ einhergeht, nicht wirklich verhindert hätte.

Mißverständnisse(?) von „Open Source“

Die Open-Source-Definition ist eindeutig genug, und es ist klar, daß typische nicht-freie Programme ihr nicht genügen. Sie vermuten jetzt wahrscheinlich, daß „Open-Source-Unternehmen“ ein Unternehmen bezeichnet, dessen Produkte freie Software (oder nahe daran) sind, nicht wahr? Leider versuchen viele Unternehmen, dem Begriff eine andere Bedeutung zu geben.

Auf der Konferenz „Open Source Developers Day“ im August 1998 sagten viele der eingeladenen kommerziellen Entwickler, daß sie nur einen Teil ihrer Arbeit zu freier Software (oder „Open Source“) machen würden. Ihr Kerngeschäft sei es, proprietäre Zusätze (Software oder Handbücher) zu entwickeln, um diese dann den Benutzern freier Software zu verkaufen. Sie forderten uns auf, dies als legitim zu betrachten – als Teil unserer Gemeinschaft –, weil ein Teil des Geldes für die Entwicklung freier Software gespendet wird.

In Wirklichkeit bemühen sich diese Unternehmen darum, das Gütesiegel „Open Source“ für ihre proprietären Software-Produkte zu erlangen – obwohl diese gar keine „Open-Source-Software“ sind –, weil diese irgendeinen Bezug zu freier Software haben oder weil dasselbe Unternehmen auch eine freie Software pflegt. (Ein Unternehmensgründer sagte ganz explizit, daß sie in das freie Software-Paket, das sie unterstützen, so wenig Arbeit hineinsteckten, wie es die Gemeinschaft noch tolerieren würde.)

Über die Jahre haben viele Unternehmen zur Entwicklung freier Software beigetragen. Manche dieser Unternehmen entwickelten in erster Linie nicht-freie Software, aber diese beiden Aktivitäten waren getrennt; dadurch konnten wir die nicht-freien Produkte des Unternehmens ignorieren und mit ihm gemeinsam an Freie-Software-Projekten arbeiten. Wir konnten uns dann ehrlich bei dem Unternehmen für seinen Beitrag zur freien Software bedanken, ohne darüber zu reden, was das Unternehmen ansonsten tat.

Dasselbe können wir bei diesen neuen Unternehmen nicht machen, weil diese uns nicht lassen. Sie laden die Öffentlichkeit aktiv ein, alle ihre Aktivitäten in einen Topf zu werfen. Sie wollen, daß wir ihre nicht-freie Software genauso positiv betrachten wie einen echten Beitrag zu freier Software, obwohl sie keiner ist. Sie präsentieren sich selbst als „Open-Source-Unternehmen“ und hoffen, daß wir kuschelig-warme Gedanken für sie entwickeln und diese mit einem kuschelig-laschen Verstand anwenden werden.

Diese manipulative Praxis wäre nicht weniger schädlich, wenn man dafür den Begriff „freie Software“ verwenden würde. Allerdings verwenden Unternehmen den Begriff „freie Software“ nicht in dieser Weise; vielleicht macht seine Verbindung mit Idealismus den Begriff ungeeignet. Der Begriff „Open Source“ öffnete dieser Praxis die Tür.

Ende 1998, auf einer Messe zu dem Betriebssystem, das häufig „Linux“ genannt wird, war ein Direktor eines bekannten Software-Unternehmens als Sprecher geladen – vermutlich wegen der Entscheidung seines Unternehmens, dieses Betriebssystem zu „unterstützen“. Leider besteht dessen Art der „Unterstützung“ darin, nicht-freie Software herauszubringen, die auf dem Betriebssystem läuft – mit anderen Worten: darin, unsere Gemeinschaft als Markt zu erschließen, aber nicht zu ihr beizutragen.

Er sagte: „Wir werden keineswegs unser Produkt zu Open Source machen, aber vielleicht zu ‚internem‘ Open Source. Wenn wir unseren Support-Mitarbeitern Zugriff auf den Quelltext geben, können diese für die Kunden Fehler beheben, und wir können ein besseres Produkt und besseren Service anbieten.“ (Dies ist kein exaktes Zitat, weil ich seine Worte nicht aufgeschrieben habe, aber es trifft das Wesentliche.)

Zuhörer sagten mir danach: „Er trifft einfach nicht den Punkt.“ Aber ist das wirklich so? Welchen Punkt hat er nicht getroffen?

Den Zielpunkt der Open-Source-Bewegung hat er nicht verfehlt. Diese Bewegung sagt nicht, daß die Anwender Freiheit haben sollten, sondern nur, daß es eine schnellere und bessere Produktentwicklung fördert, wenn man mehr Leuten erlaubt, den Quelltext einzusehen und zu verbessern. Diesen Punkt hat der Direktor voll und ganz begriffen. Er wollte den Open-Source-Ansatz nicht zur Gänze – einschließlich Anwender – umsetzen und überlegte, ihn teilweise – innerhalb der Firma – zu implementieren.

Der Punkt, den er verfehlt hat, ist der Punkt, den „Open Source“ von der Anlage her gar nicht aufwerfen soll: den Punkt, daß Benutzer Freiheit *verdienen*.

Den Gedanken der Freiheit zu verbreiten, ist eine große Aufgabe, die Ihre Mitarbeit benötigt. Deshalb halten wir im GNU-Projekt an dem Begriff „freie Software“ fest, damit wir bei dieser Aufgabe helfen können. Wenn Sie das Gefühl haben, daß Freiheit und die Gemeinschaft für sich selbst genommen wichtig sind – nicht nur

wegen der Bequemlichkeiten, die sie uns bringen –, dann schließen Sie sich bitte uns an und verwenden Sie den Begriff „freie Software“.

* * *

Joe Barr hat einen Artikel Live and let license geschrieben, der seine Ansichten zu diesem Thema wiedergibt.

Ein Fachartikel über die Motivation der Entwickler freier Software von Lakhani und Wolf besagt, daß ein nennenswerter Anteil durch den Gedanken motiviert wird, daß Software frei sein sollte. Zu diesem Ergebnis kamen sie trotz der Tatsache, daß sie Software-Entwickler bei SourceForge befragt haben – einem Betrieb, der die Anschauung nicht teilt, daß es sich hierbei um eine ethische Frage handelt.

* * *

Copyright-Notiz des englischsprachigen Originals:

Copyright © 1998, 1999, 2000, 2001 Free Software Foundation, Inc., 51 Franklin St, Fifth Floor, Boston, MA 02110, USA

Verbatim copying and distribution of this entire article is permitted in any medium without royalty provided this notice is preserved.

Übersetzung:

Es ist gebührenfrei gestattet, diesen Artikel als Ganzes und unverändert in beliebigen Medien zu kopieren und weiterzugeben, sofern dieser Hinweis erhalten bleibt.

GNU Lesser PUBLIC LISENZ Version 3

Version 3

Version 2.1

Warum man die Lesser GPL nicht für die nächste Bibliothek verwenden sollte

Bitte beachten Sie für weitere allgemeine Empfehlungen über die Wahl einer Lizenz für Ihr Werk unseren Lizenzratgeber.

Das GNU-Projekt verfügt über zwei wesentliche Lizenzen zur Verwendung mit Bibliotheken. Eine ist die GNU Lesser General Public License (LGPL), die andere ist die normale GNU General Public License (GPL). Die Wahl der Lizenz macht einen großen Unterschied: mithilfe der GNU LGPL wird die Nutzung der Bibliothek in proprietären Programmen erlaubt, mithilfe der GNU GPL ist sie ausschließlich für freie Programme nutzbar.

Welche Lizenz am besten für eine bestimmte Bibliothek geeignet ist, ist eine Frage der Strategie und situationsabhängig. Derzeit werden die meisten GNU-Bibliotheken von der GNU LGPL abgedeckt, und das bedeutet, es wird nur eine dieser zwei Strategien verwendet, die andere wird vernachlässigt. Deshalb suchen wir nun weitere Bibliotheken, um sie **unter der normalen GNU GPL** freizugeben.

Entwickler proprietärer Software haben den Vorteil des Geldes; Entwickler freier Software müssen Vorteile für einander schaffen. Das Verwenden der GNU GPL für eine Bibliothek gibt Freie-Software-Entwicklern gegenüber proprietären Entwicklern einen Vorteil: eine Bibliothek, die sie nutzen können, während proprietäre Entwickler sie nicht nutzen können.

Die Verwendung der normalen GNU GPL ist nicht für jede Bibliothek vorteilhaft. Es gibt Gründe, in denen es in bestimmten Fällen besser sein kann, die GNU LGPL zu verwenden. Häufigster Fall ist, wenn die Funktionen einer freien Bibliothek für proprietäre Software durch andere Bibliotheken leicht zugänglich sind. In diesem Fall kann die Bibliothek freier Software keinen besonderen Vorteil geben, so dass es besser ist für diese Bibliothek die GNU LGPL zu verwenden.

Deshalb haben wir die GNU LGPL für die GNU C-Bibliothek eingesetzt. Immerhin gibt es viele andere C-Bibliotheken. Mithilfe der GNU GPL wären Entwickler proprietärer Software gezwungen gewesen, eine andere zu verwenden - kein Problem für sie, nur für uns.

Wenn eine Bibliothek jedoch eine signifikante einzigartige Fähigkeit enthält, wie beispielsweise **GNU Readline**, ist das etwas ganz anderes. Die Readline-Bibliothek implementiert Eingabeergänzung und -verlauf für interaktive Programme, und das ist eine woanders nicht allgemein verfügbare Funktion. Die Freigabe unter GNU GPL - und damit die ausschließliche Nutzung in freien

Programmen - gibt unserer Gemeinschaft einen richtigen Schub. Mindestens ein Anwendungsprogramm ist heute insbesondere freie Software, weil das für die Nutzung von Readline notwendig war.

Wenn wir eine Auswahl leistungsfähiger, unter GPL lizenzierten Bibliotheken ansammeln, die keine parallelen für proprietäre Software haben, liefern sie eine Reihe nützlicher Module, die als Bausteine in neuen freien Programmen dienen. Dies wird ein signifikanter Vorteil für die weitere Entwicklung freier Software sein, und einige Projekte werden sich dazu entscheiden, Software frei zu machen, um diese Bibliotheken zu nutzen. Universitäre Projekte können leicht beeinflusst werden. Heutzutage, wo Unternehmen beginnen daran zu denken Software frei zu machen, können sogar einige kommerzielle Projekte auf diese Weise beeinflusst werden.

Entwickler proprietärer Software - bestrebt, dem freien Wettbewerb einen wichtigen Vorteil zu versagen - werden versuchen Autoren davon zu überzeugen, keine GPL-lizenzierten Bibliotheken der Sammlung beizutragen. Beispielsweise könnten sie an das Ego appellieren, wenn sie *mehr Benutzer für diese Bibliothek* versprechen, wenn wir sie den Quellcode in proprietären Softwareprodukten verwenden lassen. Popularität ist verlockend und für einen Entwickler einer Bibliothek die Idee leicht zu rationalisieren, dass die Erhöhung der Popularität dieser einen Bibliothek zu steigern das ist, was die Gemeinschaft vor allem braucht.

Aber wir sollten nicht auf diese Versuchungen hören, weil wir viel mehr erreichen können, wenn wir zusammenstehen. Wir Freie-Software-Entwickler sollten einander unterstützen. Durch die Freigabe von Bibliotheken, die nur auf freie Software begrenzt sind, helfen wir einander, dass die eigenen Freie-Software-Pakete die proprietären Pendanten übertreffen. Die gesamte Freie-Software-Bewegung wird mehr Popularität haben, weil sich freie Software als Ganzes besser gegen den Wettbewerb abheben wird.

FAQ - Question and Answer

1.1. Basic questions about the GNU Project, the Free Software Foundation, and its licenses

2.1. General understanding of the GNU licenses

3.1. Using GNU licenses for your programs

4.1. Distribution of programs released under the GNU licenses

5.1. Using programs released under the GNU licenses when writing other programs

6.1. Combining work with code released under the GNU licenses

7.1. Questions about violations of the GNU licenses

1.1. Basic questions about the GNU Project, the Free Software Foundation, and its licenses

1.1.1. What does GPL stand for?

1.1.2. Does free software mean using the GPL?

1.1.3. Why should I use the GNU GPL rather than other free software licenses?

1.1.4. Does all GNU software use the GNU GPL as its license?

1.1.5. Does using the GPL for a program make it GNU software?

1.1.6. Can I use the GPL for something other than software?

1.1.7. Why don't you use the GPL for manuals?

1.1.8. Are there translations of the GPL into other languages?

1.1.9. Why are some GNU libraries released under the ordinary GPL rather than the Lesser GPL?

1.1.10. Who has the power to enforce the GPL?

1.1.11. Why does the FSF require that contributors to FSF-copyrighted programs assign copyright to the FSF? If I hold copyright on a GPLed program, should I do this, too? If so, how?

1.1.12. Can I modify the GPL and make a modified license?

1.1.13. Why did you decide to write the GNU Affero GPLv3 as a separate license?

2.1. General understanding of the GNU licenses

2.1.1. Why does the GPL permit users to publish their modified versions?

2.1.2. Does the GPL require that source code of modified versions be posted to the public?

2.1.3. Can I have a GPL-covered program and an unrelated nonfree program on the same computer?

2.1.4. If I know someone has a copy of a GPL-covered program, can I demand they give me a copy?

2.1.5. What does "written offer valid for any third party" mean in GPLv2? Does that mean everyone in the world can get the source to any GPLed program no matter what?

- 2.1.6. *The GPL says that modified versions, if released, must be "licensed to all third parties."
Who are these third parties?*
- 2.1.7. *Does the GPL allow me to sell copies of the program for money?*
- 2.1.8. *Does the GPL allow me to charge a fee for downloading the program from my distribution site?*
- 2.1.9. *Does the GPL allow me to require that anyone who receives the software must pay me a fee and/or notify me?*
- 2.1.10. *If I distribute GPLed software for a fee, am I required to also make it available to the public without a charge?*
- 2.1.11. *Does the GPL allow me to distribute a copy under a nondisclosure agreement?*
- 2.1.12. *Does the GPL allow me to distribute a modified or beta version under a nondisclosure agreement?*
- 2.1.13. *Does the GPL allow me to develop a modified version under a nondisclosure agreement?*
- 2.1.14. *Why does the GPL require including a copy of the GPL with every copy of the program?*
- 2.1.15. *What if the work is not very long?*
- 2.1.16. *Am I required to claim a copyright on my modifications to a GPL-covered program?*
- 2.1.17. *What does the GPL say about translating some code to a different programming language?*
- 2.1.18. *If a program combines public-domain code with GPL-covered code, can I take the public-domain part and use it as public domain code?*
- 2.1.19. *I want to get credit for my work. I want people to know what I wrote. Can I still get credit if I use the GPL?*
- 2.1.20. *Does the GPL allow me to add terms that would require citation or acknowledgment in research papers which use the GPL-covered software or its output?*
- 2.1.21. *Can I omit the preamble of the GPL, or the instructions for how to use it on your own programs, to save space?*
- 2.1.22. *What does it mean to say that two licenses are "compatible"?*
- 2.1.23. *What does it mean to say a license is "compatible with the GPL"?*
- 2.1.24. *Why is the original BSD license incompatible with the GPL?*
- 2.1.25. *What is the difference between an "aggregate" and other kinds of "modified versions"?*
- 2.1.26. *When it comes to determining whether two pieces of software form a single work, does the fact that the code is in one or more containers have any effect?*
- 2.1.27. *Why does the FSF require that contributors to FSF-copyrighted programs assign copyright to the FSF?
If I hold copyright on a GPLed program, should I do this, too?
If so, how?*
- 2.1.28. *If I use a piece of software that has been obtained under the GNU GPL, am I allowed to modify the original code into a new program, then distribute and sell that new program commercially?*
- 2.1.29. *Can I use the GPL for something other than software?*
- 2.1.30. *I'd like to license my code under the GPL, but I'd also like*

to make it clear that it can't be used for military and/or commercial uses.
Can I do this?

2.1.31. Can I use the GPL to license hardware?

2.1.32. Does prelinking a GPLed binary to various libraries on the system, to optimize its performance, count as modification?

2.1.33. How does the LGPL work with Java?

2.1.34. Why did you invent the new terms "propagate" and "convey" in GPLv3?

2.1.35. Is "convey" in GPLv3 the same thing as what GPLv2 means by "distribute"?

2.1.36. If I only make copies of a GPL-covered program and run them, without distributing or conveying them to others, what does the license require of me?

2.1.37. GPLv3 gives "making available to the public" as an example of propagation. What does this mean?
Is making available a form of conveying?

2.1.38. Since distribution and making available to the public are forms of propagation that are also conveying in GPLv3, what are some examples of propagation that do not constitute conveying?

2.1.39. How does GPLv3 make BitTorrent distribution easier?

2.1.40. What is tivoization? How does GPLv3 prevent it?

2.1.41. Does GPLv3 prohibit DRM?

2.1.42. Does GPLv3 require that voters be able to modify the software running in a voting machine?

2.1.43. Does GPLv3 have a "patent retaliation clause"?

2.1.44. In GPLv3 and AGPLv3, what does it mean when it says "notwithstanding any other provision of this License"?

2.1.46. In AGPLv3, what counts as "interacting with [the software] remotely through a computer network"?

2.1.47. How does GPLv3's concept of "you" compare to the definition of "Legal Entity" in the Apache License 2.0?

2.1.48. In GPLv3, what does "the Program" refer to?
Is it every program ever released under GPLv3?

2.1.49. If some network client software is released under AGPLv3, does it have to be able to provide source to the servers it interacts with?

2.1.50. or software that runs a proxy server licensed under the AGPL, how can I provide an offer of source to users interacting with that code?

3.1. Using GNU licenses for your programs

4.1. Distribution of programs released under the GNU licenses

5.1. Using programs released under the GNU licenses when writing other programs

6.1. Combining work with code released under the GNU licenses

7.1. Questions about violations of the GNU licenses

1. Basic questions about the GNU Project, the Free Software Foundation, and its licenses

- 1.1. What does GPL stand for?
- 1.2. Does free software mean using the GPL?
- 1.3. Why should I use the GNU GPL rather than other free software licenses?
- 1.4. Does all GNU software use the GNU GPL as its license?
- 1.5. Does using the GPL for a program make it GNU software?
- 1.6. Can I use the GPL for something other than software?
- 1.7. Why don't you use the GPL for manuals?
- 1.8. Are there translations of the GPL into other languages?
- 1.9. Why are some GNU libraries released under the ordinary GPL rather than the Lesser GPL?
- 1.10. Who has the power to enforce the GPL?
- 1.11. Why does the FSF require that contributors to FSF-copyrighted programs assign copyright to the FSF? If I hold copyright on a GPLed program, should I do this, too? If so, how?
- 1.12. Can I modify the GPL and make a modified license?
- 1.13. Why did you decide to write the GNU Affero GPLv3 as a separate license?

1.1. What does GPL stand for?

SSSS

SSSS

1.2. Does free software mean using the GPL?

1.3. Why should I use the GNU GPL rather than other free software licenses?

1.4. Does all GNU software use the GNU GPL as its license?

1.5. Does using the GPL for a program make it GNU software?

1.6. Can I use the GPL for something other than software?

1.7. Why don't you use the GPL for manuals?

1.8. Are there translations of the GPL into other languages?

1.9. Why are some GNU libraries released under the ordinary GPL rather than the Lesser GPL?

1.10. Who has the power to enforce the GPL?

1.11. Why does the FSF require that contributors to FSF-copyrighted programs assign copyright to the FSF? If I hold copyright on a GPLed program, should I do this, too? If so, how?

1.12. Can I modify the GPL and make a modified license?

1.13. Why did you decide to write the GNU Affero GPLv3 as a separate license?

MIT-Lizenzen / MIT Licenses

Copyright (c) 2025 Jens Kallup

**Deutsch
Englisch
Französisch
Spanisch
Polnisch
Russisch**

MIT-Lizenz / MIT License

Deutsche Übersetzung (inoffiziell)

Copyright (c) 2025 Jens Kallup

Hiermit wird jeder Person, die eine Kopie dieser Software und der zugehörigen Dokumentationsdateien (die "Software") erhält, kostenlos die Erlaubnis erteilt, uneingeschränkt mit der Software zu handeln, einschließlich und ohne Einschränkung des Rechts, sie zu verwenden, zu kopieren, zu modifizieren, zusammenzuführen, zu veröffentlichen, zu verbreiten, zu unterlizenzieren und/oder zu verkaufen, und Personen, denen diese Software zur Verfügung gestellt wird, dies zu erlauben, unter den folgenden Bedingungen:

Der obige Urheberrechtshinweis und dieser Genehmigungshinweis sind in allen Kopien oder wesentlichen Teilen der Software beizufügen.

**DIE SOFTWARE WIRD OHNE JEDE AUSDRÜCKLICHE ODER IMPLIZIERTE GARANTIE
BEREITGESTELLT,
EINSCHLIESSLICH DER GARANTIEN DER MARKTGÄNGIGKEIT, DER EIGNUNG FÜR EINEN
BESTIMMTEN
ZWECK UND DER NICHTVERLETZUNG. IN KEINEM FALL SIND DIE AUTOREN ODER
COPYRIGHTINHABER
FÜR JEDLICHEN SCHADEN ODER SONSTIGE ANSPRÜCHE HAFTBAR, SEI ES AUFGRUND
EINER
VERTRAGSKLAGE, EINER UNERLAUBTEN HANDLUNG ODER ANDERWEITIG, DER AUS DER
SOFTWARE ODER
DER VERWENDUNG ODER ANDEREN GESCHÄFTEN MIT DER SOFTWARE ENTSTEHT.**

Original English Version

Copyright (c) 2025 Jens Kallup

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Traduction française (traduction non officielle)

Copyright (c) 2025 Jens Kallup

La permission est accordée, gratuitement, à toute personne obtenant une copie de ce logiciel et des fichiers de documentation associés (le "Logiciel"), de traiter dans le Logiciel sans restriction, y compris sans limitation les droits d'utiliser, de copier, de modifier, de fusionner, de publier, de distribuer, de sous-licencier et/ou de vendre des copies du Logiciel, et de permettre aux personnes à qui le Logiciel est fourni de le faire, sous réserve des conditions suivantes:

La mention de copyright ci-dessus et la présente autorisation doivent être incluses dans toutes les copies ou parties substantielles du Logiciel.

LE LOGICIEL EST FOURNI "EN L'ÉTAT", SANS GARANTIE D'AUCUNE SORTE, EXPRESSE OU IMPLICITE, Y COMPRIS MAIS SANS S'Y LIMITER AUX GARANTIES DE QUALITÉ MARCHANDE, D'ADÉQUATION À UN USAGE PARTICULIER ET D'ABSENCE DE CONTREFAÇON. EN AUCUN CAS, LES AUTEURS OU LES DÉTENTEURS DES DROITS D'AUTEUR NE PEUVENT ÊTRE TENUS RESPONSABLES DE TOUTE RÉCLAMATION, DOMMAGE OU AUTRE RESPONSABILITÉ, QUE CE SOIT DANS LE CADRE D'UN CONTRAT, D'UN DÉLIT OU AUTREMENT, DÉCOULANT DE, EN RELATION AVEC LE LOGICIEL OU L'UTILISATION OU AUTRES RELATIONS DANS LE LOGICIEL.

Traducción al español (traducción no oficial)

Copyright (c) 2025 Jens Kallup

Por la presente se concede permiso, libre de cargos, a cualquier persona que obtenga una copia de este software y los archivos de documentación asociados (el "Software"), para tratar con el Software sin restricción, incluyendo sin limitación los derechos a usar, copiar, modificar, fusionar, publicar, distribuir, sublicenciar y/o vender copias del Software, y para permitir a las personas a las que se les proporcione el Software hacerlo, sujeto a las siguientes condiciones:

El aviso de copyright anterior y este aviso de permiso deberán incluirse en todas las copias o partes sustanciales del Software.

EL SOFTWARE SE PROPORCIONA "TAL CUAL", SIN GARANTÍA DE NINGÚN TIPO, EXPRESA O IMPLÍCITA, INCLUYENDO PERO NO LIMITADO A LAS GARANTÍAS DE COMERCIALIZACIÓN, IDONEIDAD PARA UN PROPÓSITO PARTICULAR Y NO INFRACCIÓN. EN NINGÚN CASO LOS AUTORES O TITULARES DE LOS DERECHOS DE AUTOR SERÁN RESPONSABLES DE NINGUNA RECLAMACIÓN, DAÑOS U OTRA RESPONSABILIDAD, YA SEA EN UNA ACCIÓN DE CONTRATO, AGRAVIO O DE OTRO TIPO, QUE SURJA DE, FUERA DE O EN CONEXIÓN CON EL SOFTWARE O EL USO U OTROS TRATOS EN EL SOFTWARE.

Tłumaczenie na język polski (nieoficjalne tłumaczenie)

Copyright (c) 2025 Jens Kallup

Niniejszym udziela się każdej osobie, która otrzyma kopię tego oprogramowania i powiązanych plików dokumentacji (dalej "Oprogramowanie"), bezpłatnego pozwolenia na korzystanie z Oprogramowania bez ograniczeń, w tym bez ograniczeń do praw używania, kopiowania, modyfikowania, łączenia, publikowania, dystrybuowania, sublicencjonowania i/lub sprzedaży kopii Oprogramowania, a także zezwalania osobom, którym Oprogramowanie jest udostępniane, na to samo, z zastrzeżeniem następujących warunków:

Powyższa informacja o prawach autorskich oraz niniejsza informacja o pozwoleniu muszą być dołączone do wszystkich kopii lub istotnych części Oprogramowania.

OPROGRAMOWANIE JEST DOSTARCZANE „TAK JAK JEST”, BEZ JAKIEJKOLWIEK GWARANCJI, WYRAŻNEJ LUB DOROZUMIANEJ, W TYM MIĘDZY INNYMI GWARANCJI PRZYDATNOŚCI HANDLOWEJ, PRZYDATNOŚCI DO OKREŚLONEGO CELU ORAZ NARUSZENIA PRAW. W ŻADNYM WYPADKU AUTORZY LUB POSIADACZE PRAW AUTORSKICH NIE BĘDĄ ODPOWIEDZIALNI ZA JAKIEJKOLWIEK ROSZCZENIA, SZKODY LUB INNE ODPOWIEDZIALNOŚCI, CZY TO W RAMACH UMOWY, CZYNNYCH CZY INNYCH, WYNIKAJĄCYCH Z LUB ZWIĄZANYCH Z OPROGRAMOWANIEM LUB JEGO UŻYTKOWANIEM.

Русский перевод (неофициальный перевод)

Copyright (c) 2025 Jens Kallup

Настоящим бесплатно предоставляется разрешение любому лицу, получившему копию данного программного обеспечения и сопутствующей документации (далее "Программное обеспечение"), использовать Программное обеспечение без ограничений, включая без ограничения права на использование, копирование, изменение, слияние, публикацию, распространение, sublicenzирование и/или продажу копий Программного обеспечения, а также предоставление таких прав лицам, которым предоставляется Программное обеспечение, при соблюдении следующих условий:

Вышеуказанное уведомление об авторском праве и данное уведомление о разрешении должны быть включены во все копии или значительные части Программного обеспечения.

ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ПРЕДОСТАВЛЯЕТСЯ "КАК ЕСТЬ", БЕЗ КАКИХ-ЛИБО ГАРАНТИЙ, ЯВНЫХ ИЛИ ПОДРАЗУМЕВАЕМЫХ, ВКЛЮЧАЯ, НО НЕ ОГРАНИЧИВАЯСЬ ГАРАНТИЯМИ ТОВАРНОЙ ПРИГОДНОСТИ, ПРИГОДНОСТИ ДЛЯ ОПРЕДЕЛЕННОЙ ЦЕЛИ И НЕНАРУШЕНИЯ ПРАВ. НИ ПРИ КАКИХ ОБСТОЯТЕЛЬСТВАХ АВТОРЫ ИЛИ ПРАВООБЛАДАТЕЛИ НЕ НЕСУТ ОТВЕТСТВЕННОСТИ ЗА КАКИЕ-ЛИБО ПРЕТЕНЗИИ, УБЫТКИ ИЛИ ИНЫЕ ОБЯЗАТЕЛЬСТВА, ВОЗНИКШИЕ В РЕЗУЛЬТАТЕ ДОГОВОРА, ДЕЛИКТА ИЛИ ИНЫМ ОБРАЗОМ, ВОЗНИКШИЕ ИЗ, ИЗ-ЗА ИЛИ В СВЯЗИ С ПРОГРАММНЫМ ОБЕСПЕЧЕНИЕМ ИЛИ ИСПОЛЬЗОВАНИЕМ ИЛИ ИНЫМИ СДЕЛКАМИ С ПРОГРАММНЫМ ОБЕСПЕЧЕНИЕМ.

Free Pascal Compiler v.3.2.2

Referenz Handbuch
Dokument v.3.2.2
März 2025

Übersetzung von Jens Kallup

Liste der Tabellen
Über diesen Leitfaden
Bezeichnungen
Syntax Diagramme
Über die Sprache Pascal

1. Pascal Zeichen und Symbole

- 1.1. Symbole
- 1.2. Kommentare
- 1.3. Reservierte Schlüsselwörter
 - 1.3.1. Turbo Pascal
 - 1.3.2. Object Pascal
 - 1.3.3. Modifikationen
- 1.4. Kennzeichnungen
- 1.5. Hinweise für Direktiven
- 1.6. Zahlen
- 1.7. Bezeichner
- 1.8. Zeichenketten

2. Konstanten

- 2.1. Gewöhnliche Konstanten
- 2.2. Typisierte
- 2.3. Ressourcen Zeichenketten

3. Typen

- 3.1. Basis-Typen
 - 3.1.1. Ordinale Typen
 - 3.1.2. Ganze Zahlen (Integer)
 - 3.1.3. Boolesche Typen
 - 3.1.4. Aufzählungen
 - 3.1.5. Untermengen
 - 3.1.6. Zeichen
- 3.2. Zeichen-Typen
 - 3.2.1. Char oder AnsiChar
 - 3.2.2. WideChar
 - 3.2.3. Sonstige
 - 3.2.4. Einzel-Byte Zeichenketten
 - 3.2.4.1. ShortString
 - 3.2.4.2. AnsiString
 - 3.2.4.3. Zeichen-Code Umwandlung
 - 3.2.4.4. RawByteString
 - 3.2.4.5. UTF8String
 - 3.2.5. Multi-Byte Zeichenketten
 - 3.2.5.1. UnicodeString
 - 3.2.5.2. WideString

- 3.2.6. Konstante Zeichenketten
- 3.2.7. Nullterminierende Zeichenketten (PChar)
- 3.2.8. Zeichenketten-Größen
- 3.3. Strukturierte Typen
 - 3.3.1. Gepackte Struktur-Typen
 - 3.3.2. Array's
 - 3.3.2.1. Statische Array's
 - 3.3.2.2. Dynamische Array's
 - 3.3.2.3. Typen-Kompatibilität dynamischer Array's
 - 3.3.2.4. Constuctor dynamischer Array's
 - 3.3.2.5. Feldkonstanten-Ausdrücke dynamischer Array's
 - 3.3.2.6. Packen und Entpacken eines Array's
 - 3.3.3. Record's
 - 3.3.3.1. Layout und Größe
 - 3.3.3.2. Bemerkungen und Beispiele
 - 3.3.4. Mengen-Typen
 - 3.3.6. Datei-Typen
- 3.4. Zeiger
- 3.5. Forward-Deklarationen
- 3.6. Prozedur-Typen
- 3.7. Variant's
 - 3.7.1. Definition
 - 3.7.1. Variant's in Zuweisungen und Ausdrücken
 - 3.7.3. Variant's im Interface-Teil
- 3.8. Alias-Typen
- 3.9. Verwaltete Typen
- 4. Variablen
 - 4.1. Definition
 - 4.2. Erklärung
 - 4.3. Geltungsbereich
 - 4.4. Initialisierte Variablen
 - 4.5. Initialisierte Variablen mit Standard-Wert
 - 4.6. Thread-Variablen
 - 4.7. Eigenschaften
- 5. Objekte
 - 5.1. Deklaration
 - 5.2. Abstrakte und Sealed Objekte
 - 5.3. Felder
 - 5.4. Klassen oder statische Felder
 - 5.5. Constructor und Destructor
 - 5.6. Methoden
 - 5.6.1. Erklärung
 - 5.6.2. Methoden-Aufruf
 - 5.6.2.1. Statische Methoden
 - 5.6.2.2. Virtuelle Methoden
 - 5.6.2.3. Abstrahierte Methoden
 - 5.6.2.4. Klassen-Methoden oder statische Methoden
 - 5.7. Sichtbarkeit
- 6. Klassen
 - 6.1. Klassen-Definitionen
 - 6.2. Abstrakte und Sealed Klassen
 - 6.3. Normale und statische Felder
 - 6.3.1. Normalisierte Felder / Variablen

- 6.3.2. Klassen-Felder / Variablen
- 6.4. Klassen - CTOR
- 6.5. Klassen - DTOR
- 6.6. Methoden
 - 6.6.1. Erklärung
 - 6.6.2. Aufrufen
 - 6.6.3. Virtuelle Methoden
 - 6.6.4. Klassen - Methoden
 - 6.6.5. Klassen CTOR und DTOR
 - 6.6.6. Statische Klassen - Methoden
 - 6.6.7. Nachrichten - Methoden
 - 6.6.8. Vererbung
- 6.7. Eigenschaften
 - 6.7.1. Definition
 - 6.7.2. Indezierte Eigenschaften
 - 6.7.3. Array basierte Eigenschaften
 - 6.7.4. Standard - Eigenschaften (public)
 - 6.7.5. Veröffentlichte - Eigenschaften (published)
 - 6.7.6. Speicherinformationen
 - 6.7.7. Eigenschaften überschreiben und neu deklarieren
- 6.8. Klassen - Eigenschaften
- 6.9 Verschachtelte Typen, Konstanten, und Variablen
- 7. Schnittstellen (Interface's)
 - 7.1. Definition
 - 7.2. Identifikation
 - 7.3. Implementierung
 - 7.4. Vererbung
 - 7.5. Delegation
 - 7.6. COM
 - 7.7. CORBA und andere Schnittstellen
 - 7.8. Referenzzählung
- Generics
 - 8.1. Einführung
 - 8.2. Get'ter Typ - Definition
 - 8.3. Typen - Spezialisierung
 - 8.4. Einschränkungen
 - 8.5. Kompatibilität zu Delphi
 - 8.5.1. Syntax - Elemente
 - 8.5.2. Einschränken für Record's
 - 8.5.3. Typen - Überladung(en)
 - 8.5.4. Überlegungen für Namensbereiche
 - 8.6. Typen-Kompatibilität
 - 8.7. Verwendung der eingebauten Funktionen
 - 8.8. Überlegungen zum Geltungsbereich
 - 8.9. Operator-Überladung und Generics
- 9. Erweiterte Record's
 - 9.1. Definition
 - 9.2. Erweiterte Record-Aufzähler
 - 9.3. Record-Operationen
- 10. Klassen, Record's, und Typen-Helfer
 - 10.1 Definition
 - 10.2 Einschränkungen bei Klassen Helfer
 - 10.3 Einschränkungen bei Record Helfer

- 10.4 Überlegungen zu einfachen Helper
- 10.5 Anmerkungen zu Umfang und Lebensdauer von Record-Helper
- 10.6 Vererbung
- 10.7 Verwendung
- 11. Objektorientierte Pascal - Klassen
 - 11.1. Einführung
 - 11.2. Klassendeklarationen
 - 11.3. Formele Deklaration
 - 11.4. Instanzen zuteilen und zuordnen
 - 11.5. Protokolldefinitionen
 - 11.6. Kategorien
 - 11.7. Namensumfang und Bezeichner
 - 11.8. Selektoren
 - 11.9. Der ID Type
 - 11.10. Aufzählungen in Objective-C Klassen
- 12. Ausdrücke
 - 12.1. Ausdrucks - Syntax
 - 12.2. Funktionsaufrufe
 - 12.3. Mengen - CTOR
 - 12.4. Typ-Casting von Werten
 - 12.5. Typ-Casting von Variablen
 - 12.6. Sonstige Typ-Casting's
 - 12.7. Der @ - Operator
 - 12.8. Operatoren
 - 12.8.1. Arithmetische Operatoren
 - 12.8.2. Logische Operatoren
 - 12.8.3. Boolesche Operatoren
 - 12.8.4. Zeichenketten Operatoren
 - 12.8.5. Operatoren bei dynamischen Array's
 - 12.8.6. Mengen - Operatoren
 - 12.8.7. Relationale Operatoren
 - 12.8.8. Klassen - Operatoren
- 13. Anweisungen
 - 13.1. Einfache Anweisungen
 - 13.1.1. Zuweisungen
 - 13.1.2. Prozeduren - PROCEDURE
 - 13.1.3. Sprungs - Anweisung GOTO
 - 13.2. Strukturierte Anweisungen
 - 13.2.1. Zusammengesetzte Anweisungen
 - 13.2.2. CASE
 - 13.2.3. IF ... THEN
 - 13.2.4. FOR ... TO / DOWNT0 ... DO
 - 13.2.5. FOR .. IN .. DO
 - 13.2.6. REPEAT ... UNTIL
 - 13.2.7. WHILE ... DO
 - 13.2.8. WITH
 - 13.2.9. Ausnahmen (EXCEPT)
 - 13.3. Assembler Anweisungen
- 14. Benutzung von Funktionen und Prozeduren
 - 14.1. FUNCTION Deklarationen
 - 14.2. PROCEDURE Deklarationen
 - 14.3. Funktion Rückgabewert mittels RESULT
 - 14.4. Parameter Listen

- 14.4.1. Werte
- 14.4.2. Variablen
- 14.4.3. Ausgabe Parameter
- 14.4.4. Konstante Parameter
- 14.4.5. Offene Array's
- 14.4.6. Array of Const
- 14.4.7. Untypisierte Parameter
- 14.4.8. Verwaltete Typen und Referenzzähler
- 14.5. Überladung von Funktionen
- 14.6. Mit FORWARD deklarierte Funktionen
- 14.7. Externe Funktionen
- 14.8. Assembler Funktionen
- 14.9. Modifikatoren
 - 14.9.1. alias
 - 14.9.2. cdecl
 - 14.9.3. cppdecl
 - 14.9.4. export
 - 14.9.5. hardfloat
 - 14.9.6. inline
 - 14.9.7. interrupt
 - 14.9.8. iocheck
 - 14.9.9. local
 - 14.9.10. MS_ABI_Default
 - 14.9.11. MS_ABI_CDecl
 - 14.9.12. MWPascal
 - 14.9.13. noreturn
 - 14.9.14. nostackframe
 - 14.9.15. overload
 - 14.9.16. pascal
 - 14.9.17. public
 - 14.9.18. register
 - 14.9.19. safecall
 - 14.9.20. saveregisters
 - 14.9.21. softfloat
 - 14.9.22. stdcall
 - 14.9.23. SYSV_ABI_Default
 - 14.9.24. SYSV_ABI_CDecl
 - 14.9.25. VectorCall
 - 14.9.26. varargs
 - 14.9.27. winapi
- 14.10. Nicht unterstützte Modifikatoren
- 15. Operatoren Überladung
 - 15.1. Einleitung
 - 15.2. Operatoren - Deklarationen
 - 15.3. Operator - Zuweisung
 - 15.4. Arithmetische Operatoren
 - 15.5. Vergleichende Operatoren
 - 15.6. In Operator
 - 15.7. Logik Operatoren
 - 15.8. Auf- und Ab-Zählungs Operatoren
 - 15.9. Aufzählungs - Operator
- 16. Programme, Module und Blöcke
 - 16.1. Programme

- 16.2. Module (Unit's)
- 16.3. Namensräume
- 16.4. Abhängigkeiten von Modulen
- 16.5. Blöcke
- 16.6. Anwendungsbereiche (Scope)
 - 16.6.1. Blöcke
 - 16.6.2. Record's
 - 16.6.3. Klassen
 - 16.6.4. Unit's
- 16.7. Bibliotheken
- 17. Ausnahmen
 - 17.1. Die RAISE Anweisung
 - 17.2. Ausnahme-Behandlung und Verschachtelung
- 18. Assembler
 - 18.1. Anweisungen
 - 18.2. Prozeduren und Funktionen

Liste der Tabellen

Über dieses Handbuch

Dieses Dokument dient als Referenz für die Sprache Pascal, wie sie durch den Free Pascal Compiler implementiert ist. Es beschreibt alle Pascal-Konstrukte, die von Free Pascal unterstützt werden, und listet alle unterstützten Datentypen auf. Es vermittelt jedoch keine detaillierten Erklärungen der Pascal-Sprache: Es ist kein Anleitungsheft.

Das Ziel ist es, zu zeigen, welche Pascal-Konstrukte unterstützt werden und wo sich die Free Pascal-Implementierung von den Turbo Pascal- oder Delphi-Implementierungen unterscheidet.

Die Turbo Pascal und Delphi Pascal Compiler führten verschiedene Features in die Pascal Sprache ein. Der Free Pascal Compiler imitiert diese Compiler im entsprechenden Modus des Compilers: bestimmte Funktionen sind nur verfügbar, wenn der Compiler in den entsprechenden Modus versetzt wird.

Wenn ein bestimmtes Feature benötigt wird, ist die Verwendung des **-M** Kommandozeilenschalters oder der Direktive **{ \$MODE }** im Quelltext erforderlich. Weitere Informationen über die verschiedenen Einstellungen finden Sie im Benutzerhandbuch und im Programmiererhandbuch.

Bezeichnungen

Bezeichnungen

In diesem Dokument werden Funktionen, Typen und Variablen mit Schreibmaschinenschrift gekennzeichnet.

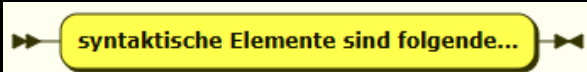
Auf Dateien wird mit einer Sans-Schrift Bezug genommen: Dateiname.

Syntax Diagramme

Diagramme zur Syntax

Alle Elemente der Sprache Pascal werden in Syntaxdiagramme erklärt. Syntaxdiagramme sind wie Flussdiagramme. Das Lesen eines Syntaxdiagramms bedeutet, von der linken Seite zur rechten Seite zu gelangen, indem man den Pfeilen folgt. Wenn die rechte Seite eines Syntaxdiagramms erreicht ist und mit einem einzelnen Pfeil endet, bedeutet, dass das Syntaxdiagramm in der nächsten Zeile fortgesetzt wird. Endet die Zeile an zwei aufeinander zeigenden Pfeilen dann ist das Diagramm beendet.

Syntaktische Elemente werden wie folgt geschrieben:



Schlüsselwörter, die genau wie im Diagramm eingegeben werden müssen:



Wenn etwas wiederholt werden kann, ist es mit einem Pfeil gekennzeichnet:

Schlüsselwort (keyword):	EBNF:
	<pre>keyword ::= "Schlüsselwort" keyword</pre>

Wenn es verschiedene Möglichkeiten gibt, werden diese in Zeilen aufgelistet:

verschiedene Möglichkeiten 1:	EBNF:
	<pre>Möglichkeiten ::= "Möglichkeit A" "Möglichkeit B" "Möglichkeit C" "..."</pre>

Beachten Sie, dass eine der Möglichkeiten leer sein kann:

verschiedene Möglichkeiten 2:	EBNF:
	<pre>Möglichkeiten ::= "Möglichkeit A" "Möglichkeit B" "Möglichkeit C" "..."</pre>

Das bedeutet, dass sowohl die erste als auch die zweite Möglichkeit optional sind. Natürlich können alle diese Elemente kombiniert und verschachtelt werden.

Über die Sprache Pascal

Die Sprache Pascal wurde ursprünglich um 1970 von Niklaus Wirth entwickelt. Seitdem hat sie sich deutlich weiterentwickelt (mit vielen Beiträgen der verschiedenen Compiler-Entwickler - insbesondere: BORLAND). Die Grundelemente wurden über die Jahre hinweg beibehalten:

- einfache Satzstruktur, recht ausführlich, aber leicht zu lesen. Ideal für den Unterricht.

- starke Typisierung.
- prozedural.
- Groß- und Kleinschreibung wird nicht berücksichtigt.
- erlaubt verschachtelte Prozeduren
- einfache integrierte Ein- und Ausgaberroutinen.

1. Pascal Zeichen und Symbole

Zeichen sind die grundlegenden lexikalischen Bausteine des Quellcodes: Sie sind die „Wörter“ der Sprache: Zeichen werden gemäß den Regeln der Programmiersprache zu Token kombiniert. Es gibt fünf Klassen von Zeichen:

- **reservierte Wörter**
Dies sind (Schlüssel)-Wörter die eine feste Bedeutung innerhalb der Sprache Pascal darstellen. Sie können nicht geändert oder neu definiert werden.
- **Bezeichner**
Dies sind Namen von Symbolen die der Programmierer definieren kann.
Sie können geändert und wiederverwendet werden.
Sie unterliegen dem Geltungsbereichsregeln der Sprache Pascal
- **Operatoren**
Dies sind in der Regel Symbole für mathematische oder andere Operationen.
Im allgemeinen betrifft dies die Symbole *, +, - und so weiter.
- **Trennzeichen**
Dies sind im allgemeine Leerzeichen
- **Konstanten**
Numerische oder Zeichenkonstanten werden verwendet, um tatsächliche Werte im Quellcode zu bezeichnen, wie beispielsweise 1 (ganzzahlige Konstante) oder 2,3 (Gleitkommakonstante) oder 'Zeichenkette'

2. Konstanten

3. Typen

4. Variablen

5. Objekte

6. Klassen

[7. Schnittstellen \(Interface's\)](#)

[8. Generics](#)

[9. Erweiterte Record's](#)

[10. Klassen, Records, und Typen-Helfer](#)

[11. Objektorientierte Pascal - Klassen](#)

[12. Ausdrücke](#)

[13. Anweisungen](#)

[14. Benutzung von Funktionen und Prozeduren](#)

[15. Operatoren - Überladung](#)

[16. Programme, Module, und Blöcke](#)

[17. Ausnahmen](#)

[18. Assembler](#)

[18.1. Anweisungen](#)

[18.2. Prozedure und Funktionen](#)